

HiDiTingV100 OpenHarmony JS 应用开发

用户指南

文档版本 00B03

发布日期 2025-07-30

前言

概述

本文档详细的描述了 HiDiTingV100 上 JS 应用控件和 API 接口的使用方法，用于指导用户开发穿戴 JS 应用。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B03	2025-07-30	- 更新 “1.2 haptobin_tool.jar 工具使用”。 - 更新 “5 OH 应用市场”。
00B02	2025-06-27	- 更新 “1.7 应用安装”。 - 更新 “5 OH 应用市场”。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 开发工具.....	1
1.1 IDE	1
1.1.1 DevEco Studio 5.0 Release.....	1
1.2 haptobin_tool.jar 工具使用.....	2
1.3 工程创建.....	3
1.3.1 DevEco Studio 5.0 Release.....	3
1.4 图形工具使用	4
1.4.1 DevEco Studio 5.0 Release.....	4
1.5 签名介绍.....	8
1.6 编译打包.....	9
1.7 应用安装.....	10
1.8 应用卸载.....	11
1.9 预置应用.....	11
2 JS 应用结构说明.....	12
2.1 文件组织.....	12
2.2 js 标签配置	14
2.3 app.js	16
2.4 语法	17
2.4.1 HML 语法参考	17
2.4.2 CSS 语法参考	23
2.4.3 JS 语法参考	27
2.5 生命周期.....	29
2.6 国际化	32
2.6.1 地区语言.....	32

3 组件	35
3.1 通用	35
3.1.1 通用属性	35
3.1.2 通用样式	36
3.1.3 通用事件	38
3.1.4 动画样式	39
3.2 容器组件	42
3.2.1 div	42
3.2.2 list	46
3.2.3 list-item	49
3.2.4 stack	50
3.2.5 swiper	51
3.3 基础组件	54
3.3.1 chart	54
3.3.2 image	62
3.3.3 image-animator	63
3.3.4 input	69
3.3.5 marquee	70
3.3.6 picker-view	71
3.3.7 progress	75
3.3.8 qrcode	77
3.3.9 slider	79
3.3.10 switch	81
3.3.11 text	82
3.4 扩展组件	86
4 接口	88
4.1 通用规则	88
4.2 通用错误码	90
4.3 基本功能	90
4.3.1 应用上下文	90
4.3.2 日志打印	91
4.3.3 页面路由	93
4.3.4 应用配置	94
4.3.5 定时器	95
4.3.6 网络	97

4.4 扩展接口..... 99

5 OH 应用市场101

A 类型说明102

Draft

插图目录

图 2-1 APP 生命周期图.....	31
图 3-1 动画样式-示例说明.....	42
图 3-2 div 组件-Flex 样式示例结果	45
图 3-3 div 组件-Flex Wrap 样式示例结果.....	46
图 3-4 list 组件-示例结果.....	49
图 3-5 stack 组件-示例结果	51
图 3-6 swiper 组件-示例结果	54
图 3-7 chart 组件-线形图示例结果.....	60
图 3-8 chart 组件-柱状图示例结果.....	62
图 3-9 image-animator 组件-示例结果.....	68
图 3-10 picker-view 组件-示例结果.....	75
图 3-11 switch 组件-示例说明	82
图 3-12 text 组件-示例结果 1	85
图 3-13 text 组件-示例结果 2	86

表格目录

表 2-1 支持的图片格式.....	14
表 2-2 js 标签配置.....	14
表 2-3 CSS 选择器.....	24
表 2-4 CSS 伪类	26
表 2-5 页面对象	28
表 2-6 应用生命周期函数	29
表 2-7 页面生命周期函数	29
表 3-1 常规属性说明	35
表 3-2 渲染属性说明	36
表 3-3 通用样式说明	36
表 3-4 通用事件说明	38
表 3-5 SwipeEvent 基础事件对象属性列表.....	39
表 3-6 动画样式说明	39
表 3-7 transform 操作说明	40
表 3-8 @keyframes 属性说明.....	41
表 3-9 div 组件样式说明.....	43
表 3-10 list 组件样式说明.....	47
表 3-11 list 组件事件说明	47
表 3-12 list 组件方法说明.....	47
表 3-13 swiper 组件属性说明.....	52

表 3-14 swiper 组件事件说明	52
表 3-15 chart 组件属性说明	54
表 3-16 ChartOptions	55
表 3-17 ChartDataset	55
表 3-18 ChartAxis	56
表 3-19 ChartSeries.....	56
表 3-20 ChartLineStyle	57
表 3-21 PointStyle	57
表 3-22 ChartLoop.....	57
表 3-23 chart 组件方法说明	58
表 3-24 image 组件属性说明	62
表 3-25 image-animator 组件属性说明	63
表 3-26 ImageFrame 说明	64
表 3-27 image-animator 组件事件说明	65
表 3-28 image-animator 组件方法说明	65
表 3-29 input 组件属性说明	69
表 3-30 input 组件样式说明	70
表 3-31 input 组件事件说明	70
表 3-32 marquee 组件属性说明	70
表 3-33 marquee 组件样式说明	71
表 3-34 picker-view 组件属性说明	71
表 3-35 文本选择器属性说明	72
表 3-36 时间选择器属性说明	72
表 3-37 picker-view 组件样式说明	72
表 3-38 文本选择器事件说明	73
表 3-39 时间选择器事件说明	73
表 3-40 progress 组件属性说明	75

表 3-41 线性进度条属性说明	76
表 3-42 弧形进度条属性说明	76
表 3-43 线性进度条样式说明	76
表 3-44 弧形进度条样式说明	76
表 3-45 qrcode 组件属性说明	78
表 3-46 qrcode 组件样式说明	78
表 3-47 二维码最小尺寸与 value 长度关系	78
表 3-48 slider 组件属性说明	80
表 3-49 slider 组件样式说明	80
表 3-50 slider 组件事件说明	80
表 3-51 ChangeEvent	80
表 3-52 switch 组件属性说明	81
表 3-53 switch 组件事件说明	81
表 3-54 text 组件样式说明	83
表 4-1 异步接口支持的回调函数	88
表 4-2 订阅接口支持的回调函数	89
表 4-3 通用错误码	90
表 4-4 AppResponse	91
表 4-5 LocaleResponse	94
表 4-6 HttpRequest	97
表 4-7 request	98
表 A-1 当前支持的颜色枚举	102

1 开发工具

- 1.1 IDE
- 1.2 haptobin_tool.jar 工具使用
- 1.3 工程创建
- 1.4 图形工具使用
- 1.5 签名介绍
- 1.6 编译打包
- 1.7 应用安装
- 1.8 应用卸载
- 1.9 预置应用

1.1 IDE

目前支持 OpenHarmony JS 应用开发的工具 IDE 分为 2.1 版本和 5.0 版本；

- 2.1 版本为早期版本，支持 API 5（已日落）。
- 5.0 版本，5.0 版本支持 API 10（推荐使用）。

1.1.1 DevEco Studio 5.0 Release

DevEco Studio For OpenHarmony（获取工具请单击[链接下载](#)，以下简称 DevEco Studio）是基于 IntelliJ IDEA Community 开源版本打造，用作开发穿戴 JS 应用。

📖 说明

当前产品使用的 IDE 版本为 DevEco Studio 5.0 Release。

1.2 haptobin_tool.jar 工具使用

背景：商用应用建议上架 release 版本安装包，本工具支持将 release hap 转换为 bin。

工具介绍：将 IDE 编译出的 release 的 hap 转为 bin 的工具。

获取路径：sdk 中的 “application/wearable/jsapp/js_sample/helper” 路径下的 fastjson-1.2.83.jar、haptobin_tool.jar

使用方法：

步骤 1 将 fastjson-1.2.83.jar、haptobin_tool.jar 放在同一目录下。

步骤 2 执行命令 “java -jar “xxx/haptobin_tool.jar” --hap-path “xxx/xxx.hap” --bin-path “xxx/xxx.bin””。

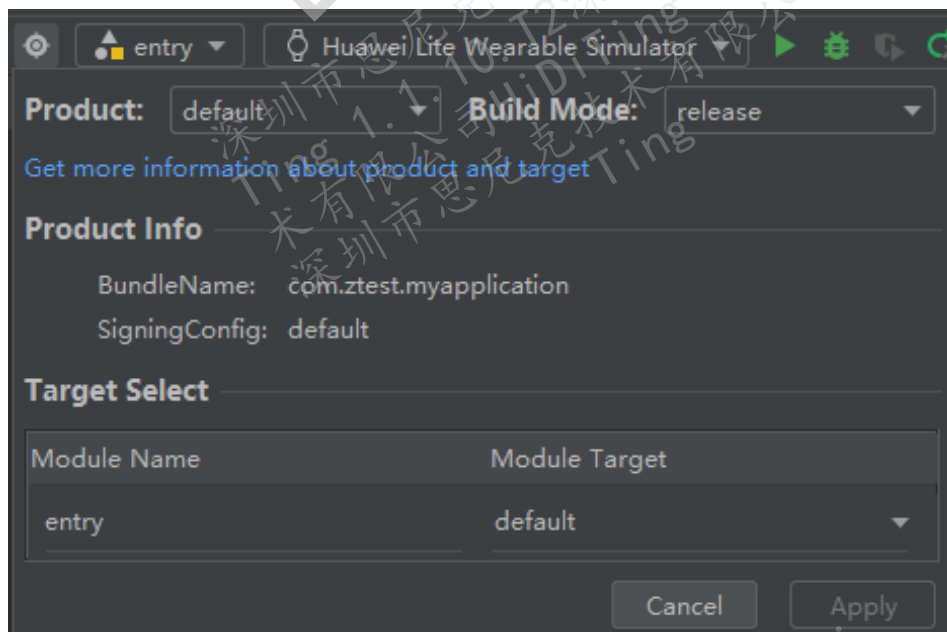
--hap-path：为 hap 包所在路径

--bin-path：所生成 bin 包所在路径

----结束

说明

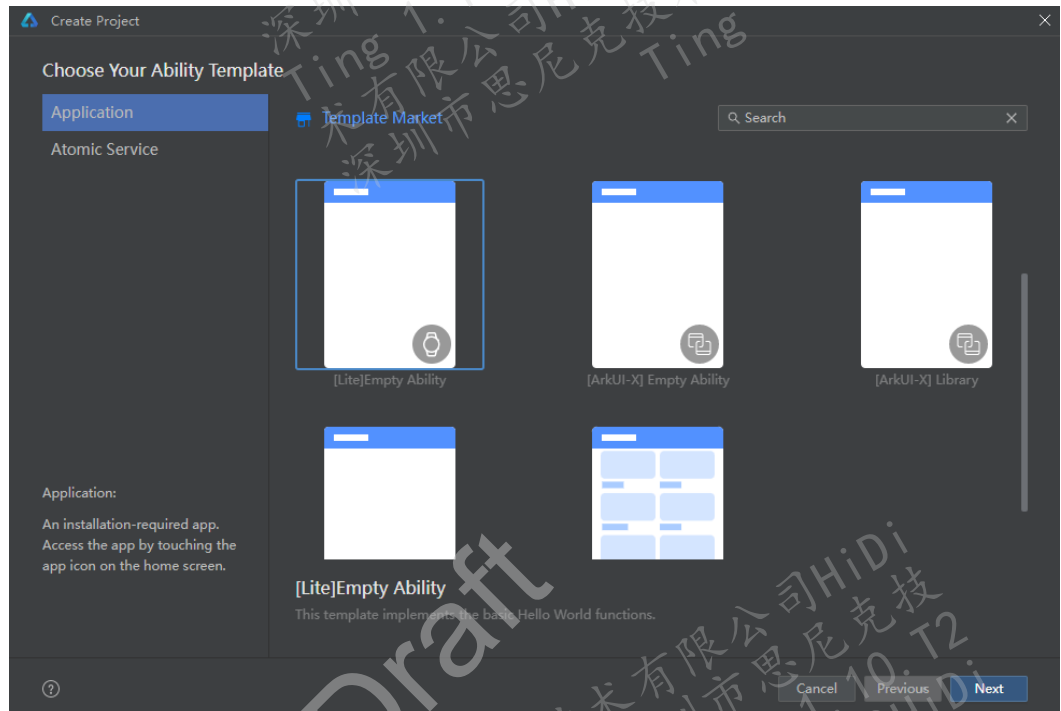
当前章节所指 hap 包是 DevEco Studio 使用 release 模式编译的*.hap 产物。如果使用 IDE 的 debug 模式，无需使用 haptobin_tool.jar。



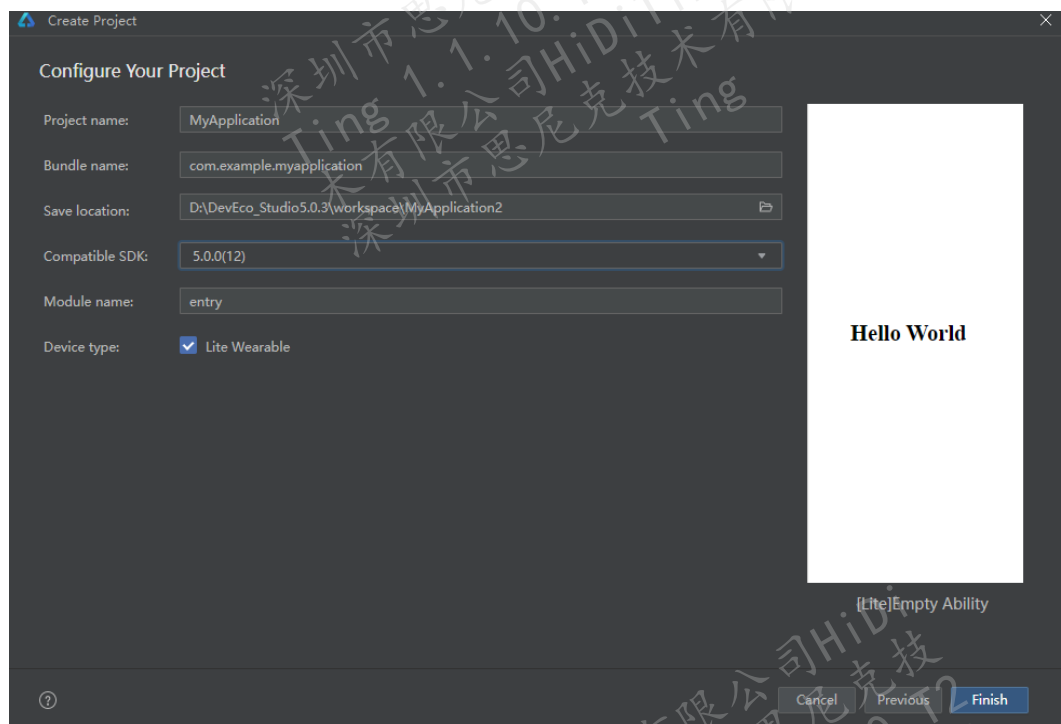
1.3 工程创建

1.3.1 DevEco Studio 5.0 Release

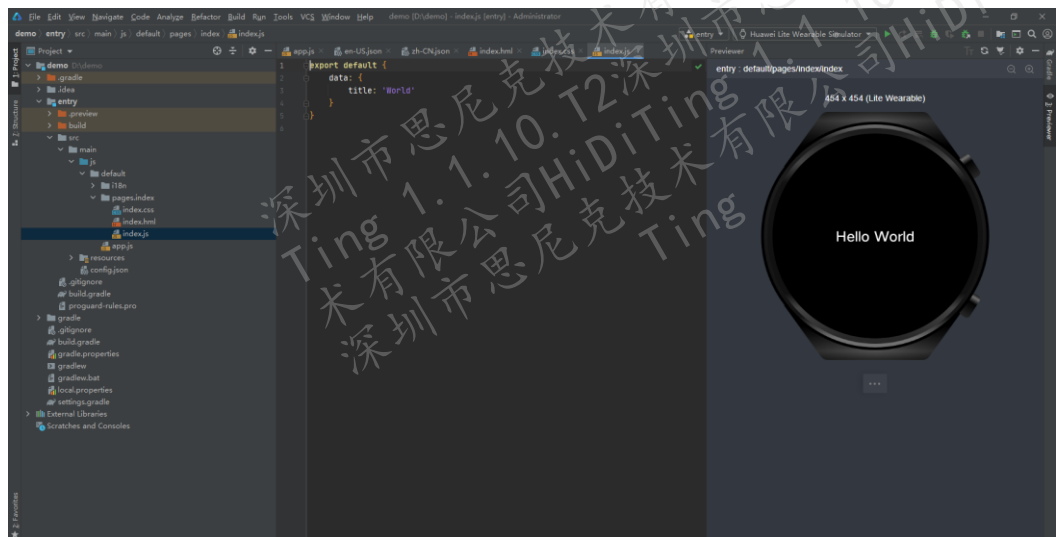
步骤 1 选择[Lite]Empty Ability(JS)工程。



步骤 2 配置项目，包括包名、工程路径、项目类型等。



步骤 3 Hello World 程序及预览显示。



说明

页面视图逻辑主要由 hml、css、js 组成，详情请参考“[2.1 文件组织](#)”。

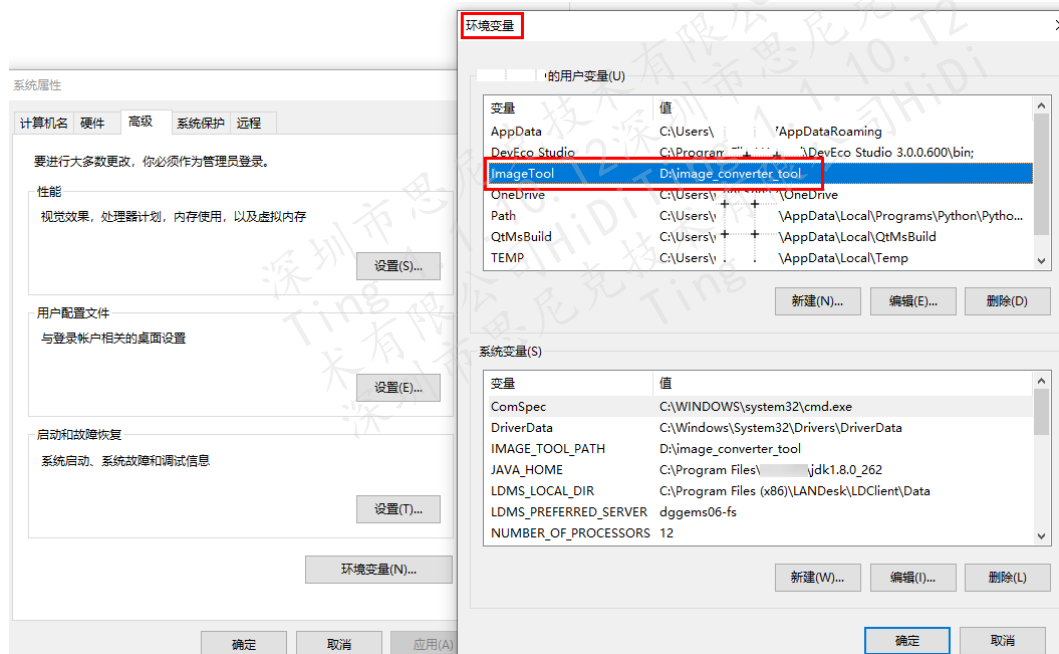
----结束

1.4 图形工具使用

1.4.1 DevEco Studio 5.0 Release

将图片资源解码成 bin 文件后打包到应用包中可以提高图片加载速度，使用步骤如下：

步骤 1 设置图像转换工具环境变量，本示例中设置如下，路径变量名为 ImageTool：



步骤 2 DevEco Studio 5.0 Release 版本在新建工程 entry 目录下的 hvigorfile.ts 脚本中添加任务如下：

```
import { legacyHapTasks ,OhosPluginId ,OhosHapContext} from '@ohos/hvigor-ohos-plugin';
import { hvigor, HvigorNode, HvigorPlugin} from '@ohos/hvigor';
import { spawn,exec,execSync} from 'child_process'
import * as fs from 'fs';
import * as path from 'path';
import { PythonShell } from 'python-shell';
function containsPng(filename: string): boolean {
    return filename.toLowerCase().includes('png') || filename.toLowerCase().includes('bmp') ||
    filename.toLowerCase().includes('jpg');
}
function copyDirectorySync(src: string, dest: string) {
    if (!fs.existsSync(dest)) {
        fs.mkdirSync(dest, { recursive: true });
    }
    const entries = fs.readdirSync(src, { withFileTypes: true });
    for (const entry of entries) {
        const srcPath = path.join(src, entry.name);
        const destPath = path.join(dest, entry.name);
        if (entry.isDirectory()) {
            copyDirectorySync(srcPath, destPath);
        } else {
            if (containsPng(srcPath)) {
                fs.copyFileSync(srcPath, destPath);
            }
        }
    }
}
```

```
    }  
  }  
}  
function createDirSync(dir: string): void {  
  if (!fs.existsSync(dir)) {  
    createDirSync(path.dirname(dir));  
    fs.mkdirSync(dir);  
  }  
}  
function delay(milliseconds: number) {  
  return new Promise<void>(resolve => {  
    setTimeout(() => {  
      resolve();  
    }, milliseconds);  
  });  
}  
function deletePngBinFiles(folderPath: string): void {  
  fs.readdirSync(folderPath).forEach((file) => {  
    const filePath = path.join(folderPath, file);  
    if (fs.lstatSync(filePath).isFile() && file.endsWith('.png.bin')) {  
      console.log(filePath);  
      fs.unlinkSync(filePath);  
    } else if (fs.lstatSync(filePath).isDirectory()) {  
      deletePngBinFiles(filePath);  
    }  
  });  
}  
async function executeCommand(dirPath: string): Promise<void> {  
  return new Promise((resolve, reject) => {  
    const myVar = process.env.ImageTool;  
    let options = { cwd: myVar };  
    console.log(myVar);  
    const result = execSync('python ' + myVar + "\\images_tool.py -p file -c config.xml -i  
"+dirPath+" -o "+dirPath, options);  
    console.log(result.toString());  
  });  
}  
export function customPlugin(): HvigorPlugin {  
  return {  
    pluginId: 'customPlugin',  
    context() {  
      return {  
        data: 'customPlugin xxx'  
      }  
    }  
  }  
}
```



```

    };
  },
  async apply(currentNode: HvigorNode): Promise<void> {
    currentNode.registerTask({
      // 任务名称
      name: `default@makepicbin`,
      // 任务执行逻辑主体函数
      async run() {
        const hapContext =
currentNode.getContext(OhosPluginId.OHOS_HAP_PLUGIN) as OhosHapContext;
        hapContext?.targets((target: default) => {
          const outputPath = target.getBuildTargetOutputPath();
          console.log(outputPath);
          const appName = 'MainAbility';
          const srcDir = `${target.projectPath}/entry/src/main/js/` + appName;
          const destDir =
`${target.projectPath}/entry/build/default/intermediates/lite_source/default/assets/js/` + appName;
          const destDir2 =
`${target.projectPath}/entry/build/default/intermediates/loader_out_lite/default/js/` + appName;
          const testDir = `${target.projectPath}/test`;
          try {
            createDirSync(destDir);
            console.log('目录创建成功');
            createDirSync(destDir2);
            console.log('目录创建成功');
            copyDirectorySync(srcDir, destDir);
            console.log('文件夹拷贝完成');
            copyDirectorySync(srcDir, destDir2);
            console.log('文件夹拷贝完成');
            console.log(`${target.projectPath}/entry/build/default/intermediates`);

executeCommand(`${target.projectPath}/entry/build/default/intermediates`);
            console.log('Command executed successfully11');

deletePngBinFiles(`${target.projectPath}/entry/build/default/intermediates`)
          } catch (error) {
            console.error('创建过程中出错:', error, destDir);
          }
        });
      },
    });
  },

```

```
// 配置前置任务依赖
dependencies: [
  `default@LegacyCompileLiteJS`,
  `default@LegacyCompileArkTS`,
  `default@LegacyGenerateLoaderJson`,
  `default@CacheNativeLibs`,
]

// 配置任务的后置任务依赖
postDependencies: [
  `default@LegacyGenerateLiteCode`,
  `default@LegacyBuildJS`,
  `default@LegacyPackageHap`,
  `default@SignHap`,
  `assembleHap`
]

});
}
};
}

export default {
  system: legacyHapTasks, /* Built-in plugin of Hvigor. It cannot be modified. */
  plugins:[customPlugin()] /* Custom plugin to extend the functionality of Hvigor.
*/
}
```

- process.env.ImageTool: [步骤 1](#) 设置的图片转换工具所在目录。

----结束

说明

图形工具源码及安装请参考 《HiDiTingV100 图形工具 使用指南》。

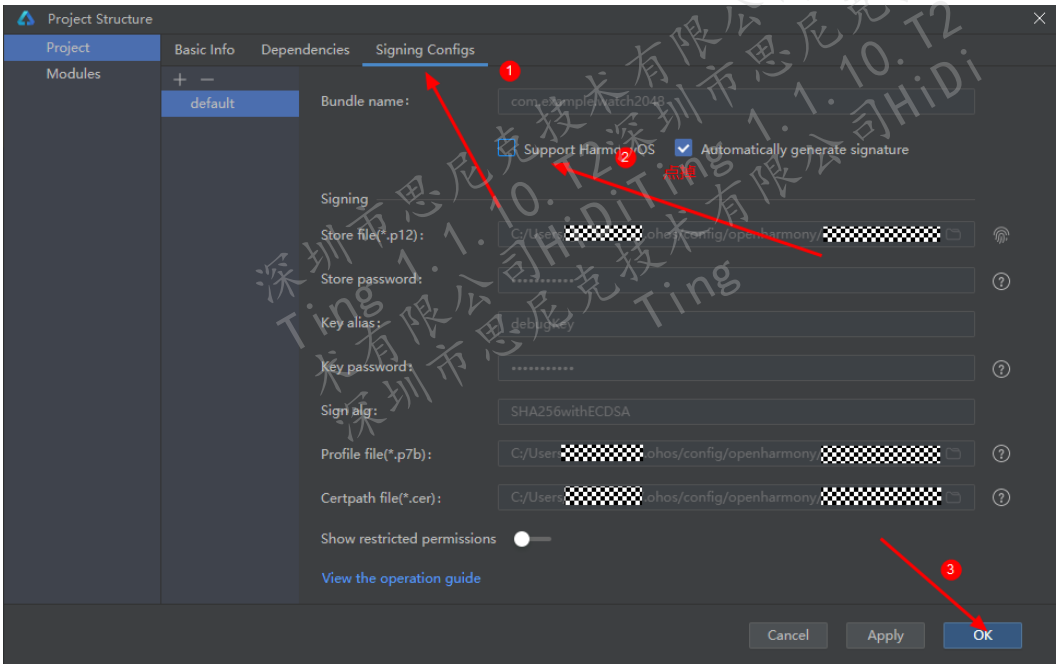
1.5 签名介绍

说明

应用签名密钥由产品厂商自行管理，解决方案不指定具体密钥。非商用调试场景，可采用调试模式关闭验签，或使用 OpenHarmony 社区签名。

OpenHarmony 社区签名方法参考如下链接：

<https://gitee.com/openharmony/docs/blob/master/zh-cn/application-dev/security/hapsigntool-guidelines.md>

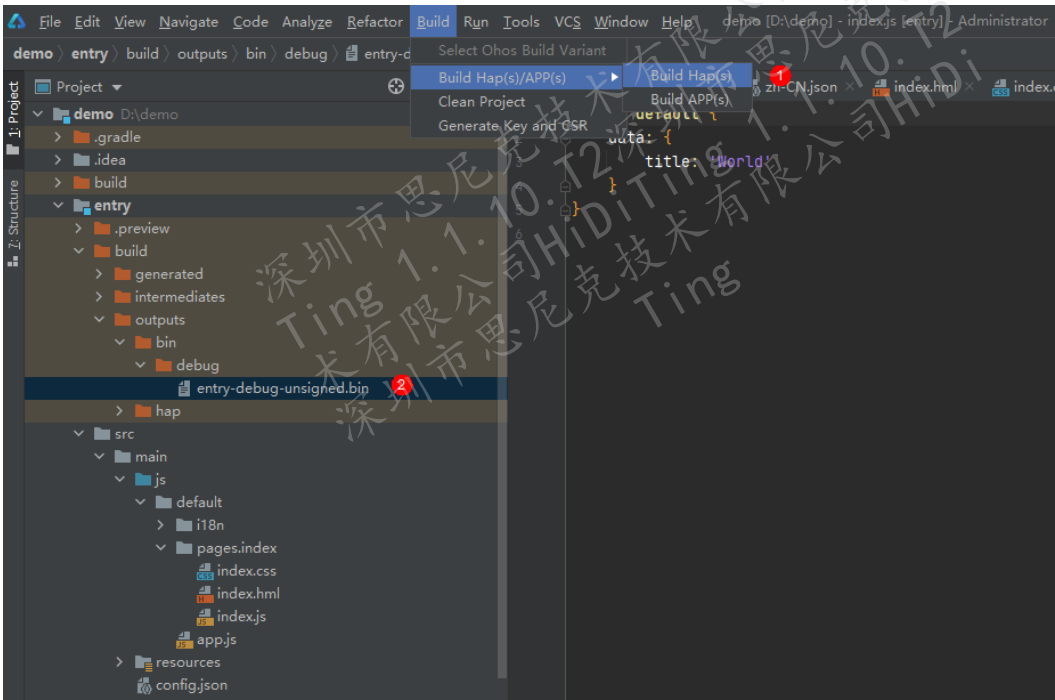


调试模式关闭验签方法如下：

- 方法一：串口执行关闭验签
AT^OHOSFWK_BM_SET=disable
- 方法二：在源码中关闭验签
修改
“open_source/openharmony/foundation/appexecfwk/appexecfwk_lite/bundlems/sign_mode.cpp” 中 isSignMode_ 的值为 false。

1.6 编译打包

按照下图中①标记的步骤构建应用包，并将在下图中②位置生成对应的应用包。



1.7 应用安装

步骤 1 将“1.6 编译打包”中生成的应用包文件上传到板端任意位置，如“/user/jsapp/sample.bin”。

说明

文件上传方式请参考《HiDiTingV100 DebugKits 工具 使用指南》“数据上传与下载”章节内容。

步骤 2 安装应用包：

1. 如果调试非签名 JSApp 时，可以在串口上执行关闭验签。对已签名应用跳过此步骤。
AT^OHOSFWK_BM_SET=disable
2. 在串口上执行应用安装，允许覆盖安装：
AT^OHOSFWK_BM_INSTALL=/user/jsapp/sample.bin
3. 如果覆盖安装时，在命令末尾添加参数“rmdata”，则会删除应用沙箱历史数据：
AT^OHOSFWK_BM_INSTALL=/user/jsapp/sample.bin rmdata

----结束

1.8 应用卸载

在串口上执行卸载：AT+OHOSFWK_BM_UNINSTALL=XXX

说明

XXX 为需要卸载应用的包名，例如：AT+OHOSFWK_BM_UNINSTALL=com.example.test。

1.9 预置应用

步骤 1 请参见《HiDiTingV100 文件系统 使用指南》的“3.2.2 开发指导”中的说明，制作文件系统镜像，将需要预置的应用放到“/user/app/system/ace/vendor/”目录下。

步骤 2 烧录镜像并重启设备，预置应用将会在设备启动时自动安装。

-----结束

说明

1. 卸载预置应用

当手动执行预置应用的卸载后，系统会在“/user/app/user/ace/etc/”目录下生成“uninstalled_delbundle.json”文件。这确保了在设备下次重启时，手动卸载的应用不会被自动重新安装。

2. 恢复预置应用

如果希望恢复之前卸载的预置应用，只需删除“/user/app/user/ace/etc/”目录下的“uninstalled_delbundle.json”文件。在删除该文件后，下次设备重启时预置应用将会重新安装。

2 JS 应用结构说明

2.1 文件组织

2.2 js 标签配置

2.3 app.js

2.4 语法

2.5 生命周期

2.6 国际化

2.1 文件组织

目录结构

JS 应用的 JS 模块（entry/src/main/js/module）的典型开发目录结构如下：

```
├── app.js
├── pages
│   ├── index
│   │   ├── index.html
│   │   ├── index.css
│   │   └── index.js
│   └── detail（可选）
│       └── detail.html
```



目录结构中文件分类如下：

- .html 结尾的 HML 模板文件，此文件用来描述当前页面的文件布局结构。
- .css 结尾的 CSS 样式文件，此文件用于描述页面样式。
- .js 结尾的 JS 文件，此文件用于处理页面和用户的交互。

各个文件夹的作用：

- pages 目录用于存放所有组件页面。
- common 目录用于存放公共资源文件，如：媒体资源和 JS 文件。
- i18n 目录用于配置不同语言场景资源内容，如应用文本词条、图片路径等资源。

说明

- i18n 是开发保留文件夹，不可重命名。
- 在使用 DevEco Studio 进行应用开发时，目录结构中的可选文件夹需要开发者根据实际情况自行创建。

文件访问规则

应用资源可通过绝对路径或相对路径的方式进行访问，本开发框架中绝对路径以 “/” 开头，相对路径以 “./” 或 “../”。具体访问规则如下：

- 引用代码文件，推荐使用相对路径，如：../common/utils.js。
- 引用资源文件，推荐使用绝对路径，如：/common/xxx.png。
- 公共代码文件和资源文件建议放在 common 下，通过以上两条规则进行访问。
- CSS 样式文件中通过 url()函数创建<url>数据类型，如：url(/common/xxx.png)。

说明

当代码文件 A 需要引用代码文件 B 时：

- 如果代码文件 A 和文件 B 位于同一目录，则代码文件 B 引用资源文件时可使用相对路径，也可使用绝对路径。
- 如果代码文件 A 和文件 B 位于不同目录，则代码文件 B 引用资源文件时必须使用绝对路径。因为 Webpack 打包时，代码文件 B 的目录会发生变化。

媒体文件格式

表2-1 支持的图片格式

格式	支持的文件类型
BMP	.bmp
JPEG	.jpg
PNG	.png

音频资源文件

音频资源文件可跟随应用一起打包编译，通过相对路径的方式进行访问，也可以预置在系统目录，使用绝对路径的方式进行访问，具体使用规则如下：

- 跟随应用打包的音频资源文件必须放置在 resources 下（默认文件夹或新增自定义子目录），如：resources/base/media/xxx.mp3。

相对路径引用时，设置播放器的 srcInner 属性，如：player.srcInner = "resources/base/media/xxx.mp3" 。

- 预置在系统目录，如：/user/music/xxx.mp3。

绝对路径引用时，设置播放器的 src 属性，如：player.src = "/user/music/xxx.mp3" 。

2.2 js 标签配置

js 标签中包含实例名称、页面路由信息如表 2-2 所示。

表2-2 js 标签配置

标签	类型	默认值	必填	说明
----	----	-----	----	----

标签	类型	默认值	必填	说明
name	string	default	是	标识 JS 实例的名字。
pages	Array	-	是	路由信息，请参见“pages”内容。

 说明

name、pages 标签配置需在“配置文件”（config.json）中的“js”标签中完成设置。

pages

定义每个页面的路由信息，每个页面由页面路径和页面名组成，页面的文件名即页面名。如：

```
{
  ...
  "pages": [
    "pages/index/index",
    "pages/detail/detail"
  ]
  ...
}
```

 说明

- 应用首页固定为“pages/index/index”。
- 页面文件名不能使用组件名称，如：text.html、button.html 等。

示例

```
{
  "app": {
    "bundleName": "com.example.player",
    "version": {
      "code": 1,
      "name": "1.0"
    },
    "vendor": "example"
  },
  "module": {
    ...
    "js": [
      {
```

```
    "name": "default",
    "pages": [
      "pages/index/index",
      "pages/detail/detail"
    ]
  },
],
"abilities": [
  {
    ...
  }
]
}
```

2.3 app.js

应用生命周期

每个应用可以在 app.js 中自定义“[2.5 生命周期](#)”的实现逻辑，以下示例仅在生命周期函数中打印对应日志：

```
// app.js
export default {
  onCreate() {
    console.info('Application onCreate');
  },

  onDestroy() {
    console.info('Application onDestroy');
  },
}
```

2.4 语法

2.4.1 HML 语法参考

HML (HarmonyOS Markup Language) 是一套类 HTML 的标记语言，通过组件、事件构建出页面的内容。页面具备数据绑定、事件绑定、列表渲染、条件渲染等高级能力。

页面结构

```
<!-- xxx.html -->
<div class="item-container">
  <text class="item-title">Image Show</text>
  <div class="item-content">
    <image src="/common/xxx.png" class="image"></image>
  </div>
</div>
```

数据绑定

```
<!-- xxx.html -->
<div onclick="changeText">
  <text> {{content[1]}} </text>
</div>
// xxx.js
export default {
  data: {
    content: ['Hello World!', 'Welcome to my world!']
  },
  changeText: function() {
    this.content.splice(1, 1, this.content[0]);
  }
}
```

说明

- 针对数组内的数据修改，请使用 splice 方法生效数据绑定变更。
- hml 文件中的 js 表达式不支持 ES6 语法。

事件绑定

事件通过'on'或者'@'绑定在组件上，当组件触发事件时会执行 JS 文件中对应的事件处理函数。

事件支持的写法如下：

- "funcName": funcName 为事件回调函数名（在 JS 文件中定义相应的函数实现）。
- "funcName(a,b)": 函数参数例如 a、b（可以为常量），或是在 JS 文件中的 data 中定义的变量（前面不用写 this）。
- 示例

```

<!-- xxx.html -->
<div class="container">
  <text class="title">{{count}}</text>
  <div class="box">
    <input type="button" class="btn" value="increase" onclick="increase" />
    <input type="button" class="btn" value="decrease" @click="decrease" />

    <!-- 传递额外参数 -->

    <input type="button" class="btn" value="double" @click="multiply(2)" />
    <input type="button" class="btn" value="decuple" @click="multiply(10)" />
    <input type="button" class="btn" value="square" @click="multiply(count)" />
  </div>
</div>

/* xxx.js */
export default {
  data: {
    count: 0
  },
  increase() {
    this.count++;
  },
  decrease() {
    this.count--;
  },
  multiply(multiplier) {
    this.count = multiplier * this.count;
  }
};

/* xxx.css */
.container {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  left: 0px;
  top: 0px;
  width: 454px;
}
```

```
height: 454px;
}
.title {
  font-size: 30px;
  text-align: center;
  width: 200px;
  height: 100px;
}
.box {
  width: 454px;
  height: 200px;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap;
}
.btn {
  width: 200px;
  border-radius: 0;
  margin-top: 10px;
  margin-left: 10px;
}
```

列表渲染

```
<!-- xxx.html -->
<div class="array-container">
  <!-- div列表渲染 -->

  <!-- 默认$item代表数组中的元素, $idx代表数组中的元素索引 -->
  <div class="div_item1" for="{{array}}" tid="id" onclick="changeText">
    <text>{{ $idx }}.{{ $item.name }}</text>
  </div>

  <!-- 自定义元素变量名称 -->
  <div class="div_item2" for="{{value in array}}" tid="id" onclick="changeText">
    <text>{{ $idx }}.{{ value.name }}</text>
  </div>

  <!-- 自定义元素变量、索引名称 -->
  <div class="div_item3" for="{{(index, value) in array}}" tid="id" onclick="changeText">
    <text>{{ index }}.{{ value.name }}</text>
  </div>
</div>
```

```
// xxx.js
export default {
  data: {
    array: [
      {id: 1, name: 'jack', age: 18},
      {id: 2, name: 'tony', age: 18},
    ],
  },
  changeText: function() {
    if (this.array[1].name === "tony"){
      this.array.splice(1, 1, {id:2, name: 'Isabella', age: 18});
    } else {
      this.array.splice(2, 1, {id:3, name: 'Bary', age: 18});
    }
  },
}

/** css */
.array-container{
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: white;
}

.div_item1{
  width: 100px;
  height: 200px;
  flex-direction: column;
  background-color: blue;
}

.div_item2{
  width: 100px;
  height: 200px;
  flex-direction: column;
  background-color: gray;
}

.div_item3{
  width: 100px;
  height: 200px;
  flex-direction: column;
```

```
background-color: red;
}

.text_item{
  color: black;
  width: 100px;
  height: 300px;
  text-align: left;
  font-size: 30px;
}
```

tid 属性主要用来加速 for 循环的重渲染，旨在列表中的数据有变更时，提高重新渲染的效率。tid 属性是用来指定数组中每个元素的唯一标识，如果未指定，数组中每个元素的索引为该元素的唯一 id。例如上述“tid=id”表示数组中的每个元素的 id 属性为该元素的唯一标识。for 循环支持的写法如下：

- for="array": 其中 array 为数组对象，array 的元素变量默认为\$item。
- for="v in array": 其中 v 为自定义的元素变量，元素索引默认为\$idix。
- for="(i, v) in array": 其中元素索引为 i，元素变量为 v，遍历数组对象 array。

📖 说明

- 数组中的每个元素必须存在 tid 指定的数据属性，否则运行时可能会导致异常。
- 数组中被 tid 指定的属性要保证唯一性，如果不是则会造成性能损耗。比如，示例中只有 id 和 name 可以作为 tid 字段，因为它们属于唯一字段。
- tid 不支持表达式。

条件渲染

条件渲染分为 2 种：if/elif/else 和 show。两种写法的区别在于：第一种写法里 if 为 false 时，组件不会在 vdom 中构建，也不会渲染，而第二种写法里 show 为 false 时虽然也不渲染，但会在 vdom 中构建；另外，当使用 if/elif/else 写法时，节点必须是兄弟节点，否则编译无法通过。实例如下：

```
<!-- xxx.html -->
<div class="container">
  <input class="btn" type="button" value="toggleShow" onclick="toggleShow"/>
  <input class="btn" type="button" value="toggleDisplay" onclick="toggleDisplay"/>
  <text class="title" if="{{visible}}"> Hello-TV </text>
  <text class="title" elif="{{display}}"> Hello-Wearable </text>
  <text class="title" else> Hello-World </text>
</div>
/* xxx.css */
```

```
.container{
  width: 100%;
  height: 100%;
  flex-direction: column;
  align-items: center;
  background-color: white;
  justify-content: center;
}
.btn{
  width: 280px;
  font-size: 26px;
  margin: 10px;
}
.title {
  width: 300px;
  height: 36px;
  color: black;
  font-size: 30px;
}
// xxx.js
export default {
  data: {
    visible: false,
    display: true,
  },
  toggleShow: function() {
    this.visible = !this.visible;
  },
  toggleDisplay: function() {
    this.display = !this.display;
  }
}
```

优化渲染优化：show 方法。

- 当 show 为 true 时，节点正常渲染。
- 当为 false 时，仅仅设置 display 样式为 none。

```
<!-- xxx.html -->
<div class="container">
  <input class="btn" type="button" value="toggle" onclick="toggle"/>
  <text style="color: red;" show="{{visible}}" > Hello World </text>
</div>
/* xxx.css */
```



```
.container{
  width: 100%;
  height: 100%;
  flex-direction: column;
  align-items: center;
  background-color: white;
  justify-content: center;
}
.btn{
  width: 280px;
  font-size: 26px;
  margin: 10px;
}
// xxx.js
export default {
  data: {
    visible: false,
  },
  toggle: function() {
    this.visible = !this.visible;
  },
}
```

说明

禁止在同一个元素上同时设置 for 和 if 属性。

2.4.2 CSS 语法参考

CSS 是描述 HML 页面结构的样式语言。所有组件均存在系统默认样式，也可在页面 CSS 样式文件中对组件、页面自定义不同的样式。

样式导入

为了模块化管理和代码复用，CSS 样式文件支持 “@import” 语句导入 css 文件。

声明样式

每个页面目录下存在一个与布局 hml 文件同名的 css 文件用来描述该 hml 页面中组件的样式，决定组件应该如何显示。

1. 内部样式，支持使用 style、class 属性来控制组件的样式。例如：

```
<!-- index.hml -->
<div class="container">
```

```
<text style="color: red">Hello World</text>
</div>
/* index.css */
.container {
  width: 100%;
  height: 100%;
  align-items: center;
  background-color: white;
  flex-direction: column;
  justify-content: center;
}
```

2. 导入外部样式文件。例如，在 common 目录中定义样式文件 style.css，并在 index.css 文件首行中进行导入：

```
/* style.css */
.title {
  font-size: 38px;
}
/* index.css */
@import '../common/style.css';
.container {
  width: 100%;
  height: 100%;
  align-items: center;
  background-color: white;
  flex-direction: column;
  justify-content: center;
}

<!-- index.html -->
<div class="container">
  <text class="title" style="color: red">Hello World</text>
</div>
```

选择器

CSS 选择器用于选择需要添加样式的元素，支持的选择器如表 2-3 所示。

表2-3 CSS 选择器

选择器	样例	样例说明
.class	.container	用于选择 class="container"的组件。

选择器	样例	样例说明
#id	#titleId	用于选择 id="titleId"的组件。
,	.title, .content	用于选择 class="title"和 class="content"的组件。

示例:

```
<!-- 页面布局xxx.html -->
<div id="containerId" class="container">
  <text id="titleId" class="title">标题</text>
  <div class="content">
    <text id="contentId">内容</text>
  </div>
</div>

/* 页面样式xxx.css */

/* 对class="title"的组件设置样式 */
.title {
  width: 150px;
  height: 70px;
  color: black;
  font-size: 38px;
}

/* 对id="contentId"的组件设置样式 */
#contentId {
  width: 150px;
  height: 70px;
  color: black;
  font-size: 38px;
}

.content{
  width: 150px;
  height: 70px;
  background-color: white;
}

/* 对所有class="title"以及class="content"的组件都设置padding为5px */
.title, .content {
  padding: 5px;
}
```

```
#containerId{
    width: 100%;
    height: 100%;
    background-color: white;
}
#titleId{
    color: green;
}
.container {
    width: 100%;
    height: 100%;
    align-items: center;
    flex-direction: column;
    justify-content: center;
}
```

伪类

CSS 伪类是选择器中的关键字如表 2-4 所示，用于指定要选择元素的特殊状态。

表2-4 CSS 伪类

名称	支持组件	说明
:active	input[type="button"]	表示被用户激活的元素，如：被用户按下的按钮。轻量级智能穿戴上伪类选择器上仅支持 background-color 和 background-image 的样式设置。
:checked	input[type="checkbox"、type="radio"]	表示 checked 属性为 true 的元素。轻量级智能穿戴上伪类选择器上仅支持 background-image 的样式设置。

设置按钮的:active 伪类可以控制被用户按下时的样式，伪类示例如下：

```
<!-- index.html -->
<div class="container">
    <input type="button" class="button" value="Button"></input>
</div>
/* index.css */
.container{
    width: 100%;
```

```
height: 100%;
align-items: center;
justify-content: center;
}
.button:active {
    background-color: #888888; /*按钮被激活时，背景颜色变为#888888 */
}
```

2.4.3 JS 语法参考

JS 文件用来定义 HML 页面的业务逻辑，支持 ECMA 规范的 JavaScript 语言。基于 JavaScript 语言的动态化能力，可以使应用更加富有表现力，具备更加灵活的设计能力。下面讲述 JS 文件的编译和运行的支持情况。

语法

支持 ES6 语法。轻量级智能穿戴支持的 ES6 语法有限，仅支持以下 ES6 语法：

let/const

arrow functions

class

default value

destructuring assignment

destructuring binding pattern

enhanced object initializer

for-of

rest parameter

template strings

- 模块声明

使用 import 方法引入功能模块：

```
import router from '@system.router';
```

- 代码引用

使用 import 方法导入 js 代码：

```
import utils from '../common/utils.js';
```

对象

表2-5 页面对象

属性	类型	说明
data	Object/Function	页面的数据模型，类型是对象或者函数，如果类型是函数，返回值必须是对象。属性名不能以\$或_开头，不要使用保留字 for、if、show、tid。
\$refs	Object	持有注册过 ref 属性的 DOM 元素或子组件实例的对象。示例请参见“ 获取 DOM 元素 ”。

获取 DOM 元素

通过\$refs 获取 DOM 元素代码示例如下：

```
<!-- index.html -->
<div class="container">
  <image-animator class="image-player" ref="animator" images="{{images}}" duration="1s"
onclick="handleClick"></image-animator>
</div>
// index.js
export default {
  data: {
    images: [
      { src: '/common/frame1.png' },
      { src: '/common/frame2.png' },
      { src: '/common/frame3.png' },
    ],
  },
  handleClick() {
    const animator = this.$refs.animator; // 获取ref属性为animator的DOM元素

    const state = animator.getState();
    if (state === 'paused') {
      animator.resume();
    } else if (state === 'stopped') {
      animator.start();
    } else {
      animator.pause();
    }
  }
}
```

```
    },  
  };  
  /** css **/  
  .container {  
    width: 100%;  
    height: 100%;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    background-color: white;  
  }  
  .image-player{  
    width: 210px;  
    height: 210px;  
    background-color: white;  
  }  
}
```

2.5 生命周期

应用生命周期

在 app.js 中可以定义如表 2-6 所示应用生命周期函数。

表2-6 应用生命周期函数

属性	类型	说明	触发时机
onCreate	() => void	应用创建	当应用创建时调用。
onDestroy	() => void	应用销毁	当应用退出时触发。

页面生命周期

在页面 JS 文件中可以定义如表 2-7 所示页面生命周期函数。

表2-7 页面生命周期函数

属性	类型	说明	触发时机
onInit	() => void	页面初始化	页面数据初始化完成时触发，只触

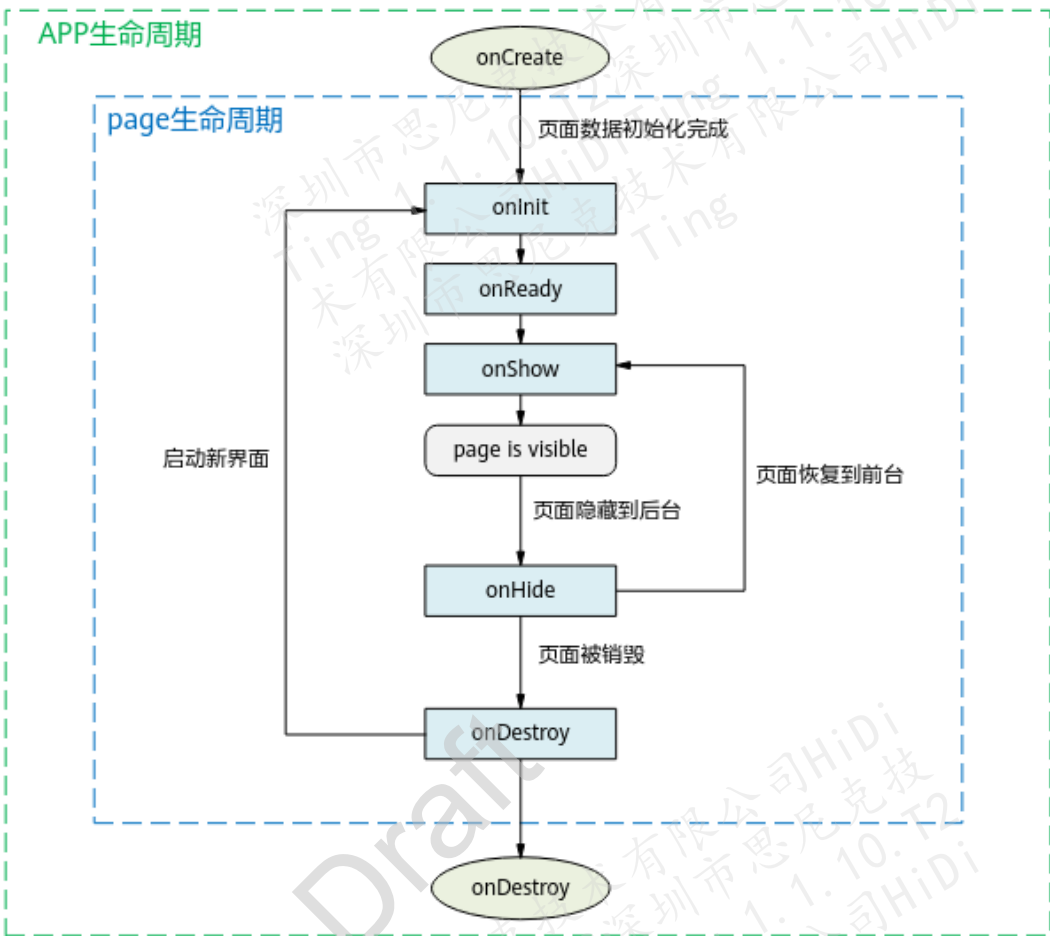
属性	类型	说明	触发时机
			发一次。
onReady	() => void	页面创建完成	页面创建完成时触发，只触发一次。
onShow	() => void	页面显示	页面显示时触发。
onHide	() => void	页面消失	页面消失时触发。
onDestroy	() => void	页面销毁	页面销毁时触发。

页面 A 的生命周期接口的调用步骤如下：

- 步骤 1 打开页面 A：onInit() → onReady() → onShow()
- 步骤 2 在页面 A 打开页面 B：onHide() → onDestroy()
- 步骤 3 从页面 B 返回页面 A：onInit() → onReady() → onShow()
- 步骤 4 退出页面 A：onHide() → onDestroy()
- 步骤 5 页面隐藏到后台运行：onHide()
- 步骤 6 页面从后台运行恢复到前台：onShow()

----结束

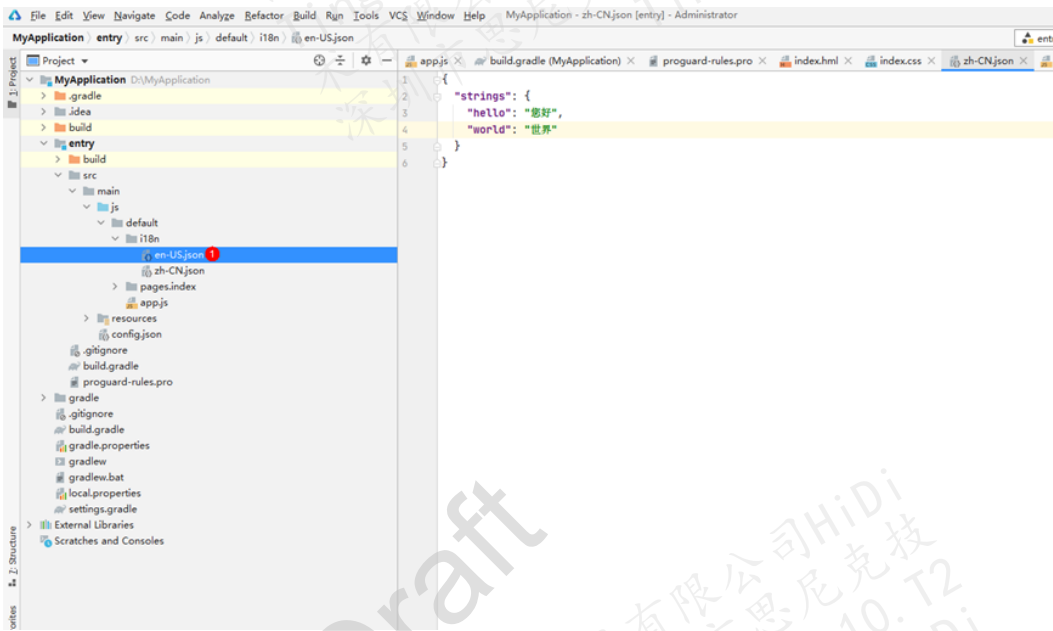
图2-1 APP 生命周期图



2.6 国际化

2.6.1 地区语言

语言场景资源内容



在 JS 应用中添加语言资源：

JSApp 工程中 i18n 目录添加应用所需语言 strings 资源内容，并在 app 中引用。json 文件的命名格式可参考：zh-CN.json、en-US.json。支持范围限制：支持的语言符合 ISO 639 标准 2 字母或 3 字母语言码，支持的国家符合 ISO 3166 标准 2 字母国家码。

示例：

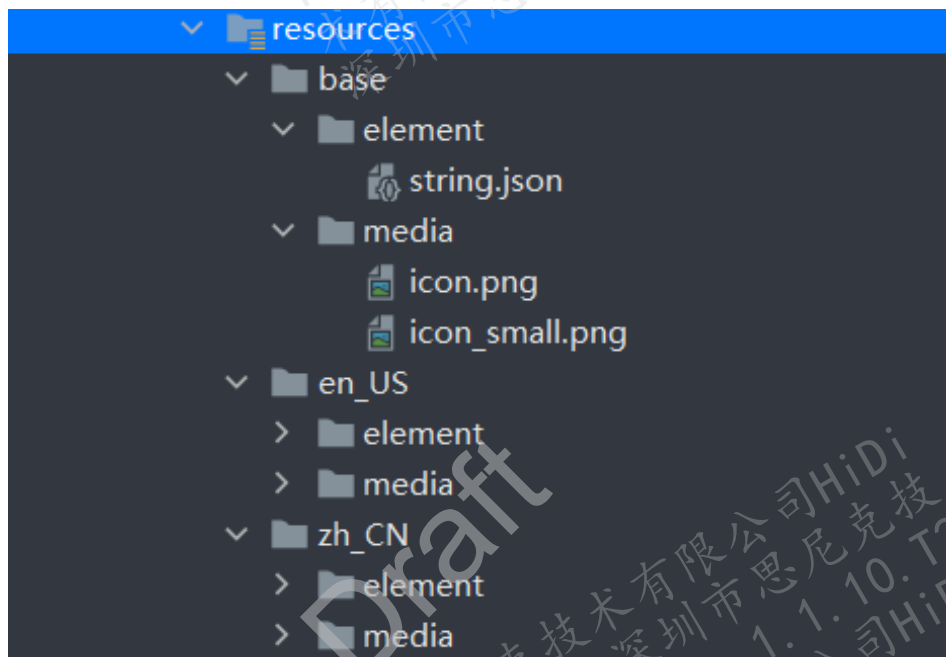
```
<!-- xxx.json -->
{
  "strings": {
    "hello": "您好",
    "world": "世界"
  }
}

<!-- xxx.html -->
<div class="container">
  <text class="title" on:click="exit">
    {{ $('strings.hello') }} {{ $('strings.world') }}
  </text>
```

</div>

应用名与图标资源：

在 entry -> src -> main -> resources 目录下，有 base、en_US、zh_CN 三个目录，分别代表默认、英语、中文资源，开发者可以根据不同语言、国家地区进行自定义。在其下级目录 element 中有 string.json 文件，修改其中 value 的值即可更改应用名称。其下级目录 media 中存储不同语言对应的图标资源文件。



说明

发布包默认支持中英，客户根据商用字库能力按需扩展。

扩展时，请根据多语言资源数量，按需调整最大值 MAX_RES_CONFIG_NUM。修改位置：
open_source/openharmony/base/global/resmgr_lite/frameworks/resmgr_lite/src/global_utils.c

切换语言：

请在产品 Settings 系统设置应用中添加语言设置，调用以下 C++接口切换系统语言，并设置默认字体样式。

示例：

```
#include "global.h"
#include "product_adapter.h"
...

// 中文
GLOBAL_ConfigLanguage("zh-CN");

// 英文
```

```
GLOBAL_ConfigLanguage("en-US");  
  
// 根据语言设置对应的JS默认字体样式，参数1是字体名称，参数2是字体大小。  
ACELite::ProductAdapter::SetDefaultFontStyle("FontName.ttf", 24);  
  
...
```

说明

语言等相关配置，需在 Settings 中管理持久化。在系统启动时，查询持久化配置，调用 global API 初始化 JSApp 语言和默认字体样式。

3 组件

- 3.1 通用
- 3.2 容器组件
- 3.3 基础组件
- 3.4 扩展组件

3.1 通用

3.1.1 通用属性

常规属性

常规属性指的是组件普遍支持的用来设置组件基本标识和外观显示特征的属性。

表3-1 常规属性说明

名称	类型	默认值	必填	说明
id	string	-	否	组件的唯一标识。
style	string	-	否	组件的样式声明。
class	string	-	否	组件的样式类，用于引用样式表。
ref	string	-	否	用来指定指向子元素的引用信息，该引用将注册到父组件的\$refs 属性对象上。

渲染属性

渲染属性指的是组件普遍支持的用来设置组件是否渲染的属性。

表3-2 渲染属性说明

名称	类型	默认值	说明
for	Array	-	根据设置的数据列表，展开当前元素。
if	boolean	-	根据设置的 boolean 值，添加或删除当前元素。
show	boolean	-	根据设置的 boolean 值，显示或隐藏当前元素。

说明

属性和样式不能混用，不能在属性字段中进行样式设置。

3.1.2 通用样式

组件普遍支持的可以在 style 或 css 中设置组件外观样式。

表3-3 通用样式说明

名称	类型	默认值	说明
width	<length> <percentage>	-	设置组件自身的宽度。 未设置时组件宽度默认为 0。
height	<length> <percentage>	-	设置组件自身的高度。 未设置时组件高度默认为 0。
padding	<length>	0	使用简写属性设置所有的内边距属性。 该属性可以有 1 到 4 个值： - 指定一个值时，该值指定四个边的内边距。 - 指定两个值时，第一个值指定上下两边的内边距，第二个指定左右两边的内边距。 - 指定三个值时，第一个指定上边的内边距，第二个指定左右两边的内边距，第三

名称	类型	默认值	说明
			个指定下边的内边距。 指定四个值时分别为上、右、下、左边的内边距（顺时针顺序）。
padding-[left top right bottom]	<length>	0	设置左、上、右、下内边距属性。
margin	<length> <percentage>	0	使用简写属性设置所有的外边距属性，该属性可以有 1 到 4 个值。 - 只有一个值时，这个值会被指定给全部的四个边。 - 两个值时，第一个值被匹配给上和下，第二个值被匹配给左和右。 - 三个值时，第一个值被匹配给上，第二个值被匹配给左和右，第三个值被匹配给下。 - 四个值时，会依次按上、右、下、左的顺序匹配（即顺时针顺序）。
margin-[left top right bottom]	<length> <percentage>	0	设置左、上、右、下外边距属性。
border-width	<length>	0	使用简写属性设置元素的所有边框宽度。
border-color	<color>	black	使用简写属性设置元素的所有边框颜色。
border-radius	<length>	-	border-radius 属性是设置元素的外边框圆角半径。
background-color	<color>	-	设置背景颜色。
opacity ⁵⁺	number	1	元素的透明度，取值范围为 0 到 1。 1：不透明。 0：完全透明。
display	string	flex	确定一个元素所产生的框的类型，可选值

名称	类型	默认值	说明
			为： - flex：弹性布局。 - none：不渲染此元素。
[left top]	<length> <percentage>	-	left top 确定元素的偏移位置。 - left 属性规定元素的左边缘。该属性定义了定位元素左外边距边界与其包含块左边界之间的偏移。 - top 属性规定元素的顶部边缘。该属性定义了一个定位元素的上外边距边界与其包含块上边界之间的偏移。

说明

通用样式都不是必填项。

3.1.3 通用事件

事件说明

- 事件绑定在组件上，当组件达到事件触发条件时，会执行 JS 中对应的事件回调函数，实现页面 UI 视图和页面 JS 逻辑层的交互。
- 事件回调函数中通过参数可以携带额外的信息，如组件上的数据对象 dataset，事件特有的回调参数。

相对于私有事件，大部分组件都可以绑定如表 3-4 所示事件。

表3-4 通用事件说明

名称	参数	说明	是否支持冒泡
click	-	点击动作触发该事件。	是
longpress	-	长按动作触发该事件。	是
swipe	表 3-5	组件上快速滑动后触发该事件。	是

说明

除上述事件外，其他事件均为非冒泡事件，如“input 的 change 事件”，请参见各个组件。

表3-5 SwipeEvent 基础事件对象属性列表

属性	类型	说明
direction	string	滑动方向，可能值有： • left：向左滑动； • right：向右滑动； • up：向上滑动； • down：向下滑动。

3.1.4 动画样式

组件支持动态的旋转、平移、缩放效果，可在 style 或 css 中设置。

表3-6 动画样式说明

名称	类型	默认值	说明
transform	string	-	请参见表 3-7。
animation-name	string	-	指定@keyframes，请参见表 3-8。
animation-delay	<time>	0	定义动画播放的延迟时间。支持的单位为 [s(秒) ms(毫秒)]，默认单位为 ms，格式为：1000ms 或 1s。
animation-duration	<time>	0	定义一个动画周期。支持的单位为 [s(秒) ms(毫秒)]，默认单位为 ms，格式为：1000ms 或 1s。 说明 轻量级智能穿戴上，动画周期最大值为 60s。 animation-duration 样式必须设置，否则时长为 0，则不会播放动画。
animation-iteration-count	number infinite	1	定义动画播放的次数，默认播放一次，可通过设置为 infinite 无限次播放。

名称	类型	默认值	说明
animation-timing-function	string	linear	描述动画执行的速度曲线，用于使动画更为平滑。 可选项有： - linear：表示动画从头到尾的速度都是相同的。 - ease-in：表示动画以低速开始，cubic-bezier(0.42, 0.0, 1.0, 1.0)。 - ease-out：表示动画以低速结束，cubic-bezier(0.0, 0.0, 0.58, 1.0)。 - ease-in-out：表示动画以低速开始和结束，cubic-bezier(0.42, 0.0, 0.58, 1.0)。
animation-fill-mode	string	none	指定动画开始和结束的状态： - none：在动画执行之前和之后都不会应用任何样式到目标上。 - forwards：在动画结束后，目标将保留动画结束时的状态（在最后一个关键帧中定义）。

表3-7 transform 操作说明

名称	类型	说明
translateX	<length>	X 轴方向平移动画属性。
translateY	<length>	Y 轴方向平移动画属性。
rotate	<deg> <rad>	旋转动画属性

 说明

轻量级智能穿戴仅支持原始大小的图片进行旋转。

表3-8 @keyframes 属性说明

名称	类型	默认值	说明
background-color	<color>	-	动画执行后应用到组件上的背景颜色。
width	<length>	-	动画执行后应用到组件上的宽度值。
height	<length>	-	动画执行后应用到组件上的高度值。
transform	string	-	定义应用在组件上的变换类型，见 表 3-6 。

对于不支持起始值或终止值缺省的情况，可以通过 from 和 to 显示指定起始和结束。
示例：

```
<!-- xxx.html -->
<div class="container">
  <div class="rect">
  </div>
</div>
/* xxx.css */
.container {
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
}
.rect{
  width: 200px;
  height: 200px;
  background-color: #f76160;
  animation-name: Go;
  animation-iteration-count: infinite;
  animation-duration: 3s;
}
@keyframes Go
{
  from {
    background-color: #f76160;
  }
}
```

```
to {  
  background-color: #09ba07;  
}  
}
```

图3-1 动画样式-示例说明



说明

@keyframes 的 from/to 不支持动态绑定。

3.2 容器组件

3.2.1 div

基础容器，用作页面结构的根节点或将内容进行分组。

子组件

支持。

属性

支持“[3.1.1 通用属性](#)”中所述属性。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-9 所示样式。

表3-9 div 组件样式说明

名称	类型	默认值	必填	说明
flex-direction	string	row	否	flex 容器主轴方向。可选项有： - column：垂直方向从上到下。 - row：水平方向从左到右。
flex-wrap	string	nowrap	否	flex 容器是单行还是多行显示，该值暂不支持动态修改。可选项有： - nowrap：不换行，单行显示。 - wrap：换行，多行显示。
justify-content	string	flex-start	否	flex 容器当前行的主轴对齐格式。可选项有： - flex-start：项目位于容器的开头。 - flex-end：项目位于容器的结尾。 - center：项目位于容器的中心。 - space-between：项目位于各行之间留有空白的容器内。 - space-around：项目位于各行之前、之间、之后都留有空白的容器内。
align-items	string	stretch ⁵⁺ flex-start ¹⁻⁴ 5+：支持 SDK 版本大于 5。 1-4：支持 SDK 版本在 1-4。	否	flex 容器当前行的交叉轴对齐格式，可选值为： - stretch：弹性元素在交叉轴方向被拉伸到与容器相同的高度或宽度。 - flex-start：元素向交叉轴起点对齐。

名称	类型	默认值	必填	说明
				- flex-end：元素向交叉轴终点对齐。 - center：元素在交叉轴居中。
display	string	flex	否	确定该元素视图框的类型，该值暂不支持动态修改。可选值为： - flex：弹性布局。 - none：不渲染此元素。

事件

支持“3.1.3 通用事件”中所述事件。

示例

- Flex 样式：

```
<!-- xxx.html -->
<div class="container">
  <div class="flex-box">
    <div class="flex-item color-primary"></div>
    <div class="flex-item color-warning"></div>
    <div class="flex-item color-success"></div>
  </div>
</div>

/* xxx.css */
.container {
  flex-direction: column;
  justify-content: center;
  align-items: center;
  width: 454px;
  height: 454px;
}
.flex-box {
  justify-content: space-around;
  align-items: center;
  width: 400px;
  height: 140px;
  background-color: #ffffff;
}
```

```
}  
.flex-item {  
  width: 120px;  
  height: 120px;  
  border-radius: 16px;  
}  
.color-primary {  
  background-color: #007dff;  
}  
.color-warning {  
  background-color: #ff7500;  
}  
.color-success {  
  background-color: #41ba41;  
}
```

图3-2 div 组件-Flex 样式示例结果



- Flex Wrap 样式

```
<!-- xxx.html -->  
<div class="container">  
  <div class="flex-box">  
    <div class="flex-item color-primary"></div>  
    <div class="flex-item color-warning"></div>  
    <div class="flex-item color-success"></div>  
  </div>  
</div>  
/* xxx.css */  
.container {  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  width: 454px;  
  height: 454px;  
}  
.flex-box {  
  justify-content: space-around;  
  align-items: center;
```

```
flex-wrap: wrap;
width: 300px;
height: 250px;
background-color: #ffffff;
}
.flex-item {
  width: 120px;
  height: 120px;
  border-radius: 16px;
}
.color-primary {
  background-color: #007dff;
}
.color-warning {
  background-color: #ff7500;
}
.color-success {
  background-color: #41ba41;
}
```

图3-3 div 组件-Flex Wrap 样式示例结果



3.2.2 list

列表包含一系列相同宽度的列表项。适合连续、多行呈现同类数据，例如图片和文本。

子组件

仅支持<3.2.3 list-item>子组件。

属性

支持“3.1.1 通用属性”中所述属性。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-10 所示样式。

表3-10 list 组件样式说明

名称	类型	默认值	必填	说明
flex-direction	string	column	否	设置 flex 容器主轴的方向，指定 flex 项如何放置在 flex 容器中，可选值为： - column：主轴为纵向。 - row：主轴为横向。 其他组件默认值为 row，在 list 组件中默认值为 column。轻量级智能穿戴不支持动态修改。

事件

除支持“3.1.3 通用事件”中所述事件外，还支持如表 3-11 所示事件。

表3-11 list 组件事件说明

名称	参数	说明
scrollend	-	列表滑动已经结束。
scrollbar	bool	是否展示 list 进度条，false 为不显示，true 为显示；组件 scrollbar 属性默认为 false。

方法

表3-12 list 组件方法说明

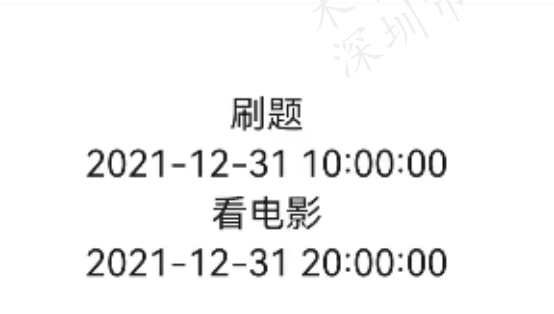
名称	参数	说明
scrollTo	{ index: number(指定位置) }	list 滑动到指定 index 的 item 位置。

示例

```
<!-- index.html -->
<div class="container">
  <list class="todo-wrapper">
    <list-item for="{{todolist}}" class="todo-item">
      <text class="todo-title">{{item.title}}</text>
      <text class="todo-title">{{item.date}}</text>
    </list-item>
  </list>
</div>
// index.js
export default {
  data: {
    todolist: [{
      title: '刷题',
      date: '2021-12-31 10:00:00',
    }, {
      title: '看电影',
      date: '2021-12-31 20:00:00',
    }],
  },
}
/* index.css */
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  left: 0px;
  top: 0px;
  width: 454px;
  height: 454px;
}
.todo-wrapper {
  width: 454px;
  height: 300px;
}
.todo-item {
  width: 454px;
  height: 80px;
  flex-direction: column;
}
```

```
}  
.todo-title {  
  width: 454px;  
  height: 40px;  
  text-align: center;  
}
```

图3-4 list 组件-示例结果



3.2.3 list-item

list-item 是[3.2.2 list](#)的子组件，用来展示列表具体 item。

子组件

支持。

属性

支持“[3.1.1 通用属性](#)”中所述属性。

样式

支持“[3.1.2 通用样式](#)”中所述样式。

事件

支持“[3.1.3 通用事件](#)”中所述事件。

示例

请参见“[示例](#)”中示例内容。

3.2.4 stack

堆叠容器，子组件按照顺序依次入栈，后一个子组件覆盖前一个子组件。

子组件

支持。

属性

支持 “3.1.1 通用属性” 中所述属性。

样式

支持 “3.1.2 通用样式” 中所述样式。

事件

支持 “3.1.3 通用事件” 中所述事件。

示例

```
<!-- xxx.html -->
<stack class="stack-parent">
  <div class="back-child bd-radius"></div>
  <div class="positioned-child bd-radius"></div>
  <div class="front-child bd-radius"></div>
</stack>
/* xxx.css */
.stack-parent {
  width: 400px;
  height: 400px;
  background-color: #ffffff;
  border-width: 1px;
  border-style: solid;
}
.back-child {
  width: 300px;
  height: 300px;
  background-color: #3f56ea;
}
.front-child {
  width: 100px;
```

```
height: 100px;
background-color: #00bfc9;
}
.positioned-child {
width: 100px;
height: 100px;
left: 50px;
top: 50px;
background-color: #47cc47;
}
.bd-radius {
border-radius: 16px;
}
```

图3-5 stack 组件-示例结果



3.2.5 swiper

滑动容器，提供切换子组件显示的能力。

子组件

支持除<list>之外的子组件。

属性

除支持“[3.1.1 通用属性](#)”中所述属性外，还支持如[表 3-13](#)所示属性。

表3-13 swiper 组件属性说明

名称	类型	默认值	必填	说明
index	number	0	否	当前在容器中显示的子组件的索引值。
loop	boolean	true	否	是否开启循环滑动。 不支持动态修改。 说明 loop 参数生效需要满足以下两个条件： - 除第一个子组件之外，剩余子组件的总长度大于等于 swiper 的长度。 - 除最后一个子组件之外，剩余子组件的总长度大于等于 swiper 的长度。
duration	number	-	否	子组件切换的动画时长。
vertical	boolean	false	否	是否为纵向滑动，纵向滑动时采用纵向的指示器。 不支持动态修改。

样式

支持 “3.1.2 通用样式” 中所述样式。

事件

表3-14 swiper 组件事件说明

名称	参数	说明
change	{ index: currentIndex }	当前显示的组件索引变化时触发该事件。

示例

```
<!-- xxx.html -->
<swiper class="container" index="{{index}}">
  <div class="swiper-item primary-item">
    <text>1</text>
  </div>
  <div class="swiper-item warning-item">
    <text>2</text>
  </div>
  <div class="swiper-item success-item">
    <text>3</text>
  </div>
</swiper>

/* xxx.css */
.container {
  left: 0px;
  top: 0px;
  width: 454px;
  height: 454px;
}
.swiper-item {
  width: 454px;
  height: 454px;
  justify-content: center;
  align-items: center;
}
.primary-item {
  background-color: #007dff;
}
.warning-item {
  background-color: #ff7500;
}
.success-item {
  background-color: #41ba41;
}

/* xxx.js */
export default {
  data: {
    index: 1
  }
}
```

图3-6 swiper 组件-示例结果



3.3 基础组件

3.3.1 chart

图表组件，用于呈现线形图、柱状图界面。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-15 所示属性。

表3-15 chart 组件属性说明

名称	类型	默认值	必填	说明
type	string	line	否	设置图表类型（不支持动态修改），可选项有： • bar：柱状图。 • line：线形图。
options	表 3-16	-	是	图表参数设置，柱状图和线形图必须设置参数设置。可以设置 x 轴、y 轴的最小值、最大值、刻度数、是否显示，线条宽度、是否平滑等。（不支持动态修改）

名称	类型	默认值	必填	说明
datasets	Array<表 3-17>	-	是	数据集合，柱状图和线形图必须设置数据集合。可以设置多条数据集及其背景色。

表3-16 ChartOptions

名称	类型	默认值	必填	说明
xAxis	表 3-18	-	是	x 轴参数设置。可以设置 x 轴最小值、最大值、刻度数以及是否显示。
yAxis	表 3-18	-	是	y 轴参数设置。可以设置 y 轴最小值、最大值、刻度数以及是否显示。
series	表 3-19	-	否	数据序列参数设置。 - 设置线的样式，如线宽、是否平滑。 - 设置线最前端位置白点的样式和大小。 说明 仅线形图支持。

表3-17 ChartDataset

名称	类型	默认值	必填	说明
background Color(depre cated)	<color>	#ff6384	否	设置线或柱的颜色（不推荐使用）。
strokeColor	<color>	#ff6384	否	线条颜色。 说明 仅线形图支持。
fillColor	<color>	#ff6384	否	填充颜色。线形图表示填充的渐变颜色。
data	Array<num ber>	-	是	设置绘制线或柱中的点集。

名称	类型	默认值	必填	说明
gradient	boolean	false	否	设置是否显示填充渐变颜色。 说明 仅线形图支持。

表3-18 ChartAxis

名称	类型	默认值	必填	说明
min	number	0	否	轴的最小值。 说明 仅线形图支持负数。
max	number	100	否	轴的最大值。 说明 仅线形图支持负数。
axisTick	number	10	否	轴显示的刻度数量。 说明 仅支持 1 ~ 20，且具体显示的效果与如下计算值有关（图的宽度所占的像素/（max-min））。 因轻量级智能穿戴为整型运行，在除不尽的情况下会有误差产生，具体的表现形式是 x 轴末尾可能会空出一段。 在柱状图中，每组数据显示的柱子数量与刻度数量一致，且柱子显示在刻度处。
display	boolean	false	否	是否显示轴。
color	<color>	#c0c0c0	否	轴颜色。

表3-19 ChartSeries

名称	类型	默认值	必填	说明
lineStyle	表 3-20	-	否	线样式设置，如线宽、是否平滑。
headPoint	表 3-21	-	否	线最前端位置白点的样式和大小。

名称	类型	默认值	必填	说明
topPoint	表 3-21	-	否	最高点的样式和大小。
bottomPoint	表 3-21	-	否	最低点的样式和大小。
loop	表 3-22	-	否	设置屏幕显示满时，是否需要重头开始绘制。

表3-20 ChartLineStyle

名称	类型	默认值	必填	说明
width	<length>	1px	否	线宽设置。
smooth	boolean	false	否	是否平滑。

表3-21 PointStyle

名称	类型	默认值	必填	说明
shape	string	circle	否	高亮点的形状。可选值为： circle：圆形。
size	<length>	5px	否	高亮点的大小。
strokeWidth	<length>	1px	否	边框宽度
strokeColor	<color>	#ff0000	否	边框颜色。
fillColor	<color>	#ff0000	否	填充颜色。
display	boolean	true	否	是否高亮显示。

表3-22 ChartLoop

名称	类型	默认值	必填	说明
margin	<length>	1	否	擦除点的个数（最新绘制的点与最老的点之间的横向距离）。 注意：轻量设备 margin 和

名称	类型	默认值	必填	说明
				topPoint/bottomPoint/headPoint 同时使用时，有概率出现 point 正好位于擦除区域的情况，导致 point 不可见，因此不建议同时使用。

样式

支持 “3.1.2 通用样式” 中所述样式。

事件

支持 “3.1.3 通用事件” 中所述事件。

方法

表3-23 chart 组件方法说明

方法	参数	说明
append	<pre>{ serial: number, // 设置要更新的线形图数据下标 data: Array<number>, // 设置新增的数据 }</pre>	往已有的数据序列中动态添加数据，根据 serial 指定目标序列，serial 为 datasets 数组的下标，从 0 开始。 注意：不会更新 datasets[index].data。仅线形图支持，按横坐标加 1 递增（与 xAxis min/max 设置相关）。

示例

- 线形图

```
<!-- xxx.html -->
<div class="container">
  <chart class="chart" type="line" ref="linechart" options="{{lineOps}}"
  datasets="{{lineData}}"></chart>
```

```
<input class="button" type="button" value="Add data" onclick="addData"/>
</div>
/* xxx.css */
.container {
  flex-direction: column;
  justify-content: center;
  align-items: center;
  width: 454px;
  height: 454px;
  background-color: white;
}
.chart {
  width: 300px;
  height: 300px;
}
.button {
  width: 280px;
  border-radius: 0px;
}
// xxx.js
export default {
  data: {
    lineData: [
      {
        strokeColor: '#0081ff',
        fillColor: '#cce5ff',
        data: [763, 550, 551, 554, 731, 654, 525, 696, 595, 628, 791, 505, 613, 575, 475,
553, 491, 680, 657, 716],
        gradient: true,
      }
    ],
    lineOps: {
      xAxis: {
        min: 0,
        max: 20,
        display: false,
      },
      yAxis: {
        min: 0,
        max: 1000,
        display: false,
      },
      series: {
```

```
lineStyle: {
  width: "5px",
  smooth: true,
},
headPoint: {
  shape: "circle",
  size: 10,
  strokeWidth: 5,
  fillColor: '#ffffff',
  strokeColor: '#007aff',
  display: true,
},
loop: {
  margin: 2,
}
},
},
addData() {
  this.$refs.linechart.append({
    serial: 0,
    data: [Math.floor(Math.random() * 400) + 400]
  })
}
```

图3-7 chart 组件 线形图示例结果



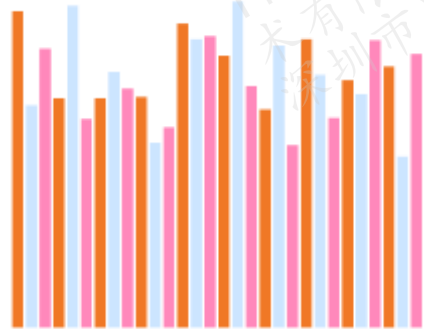
- 柱状图

```
<!-- xxx.html -->
<div class="container">
```

```
<chart class="chart" type="bar" id="bar-chart" options="{{barOps}}"
datasets="{{barData}}"></chart>
</div>
/* xxx.css */
.container {
  flex-direction: column;
  justify-content: center;
  align-items: center;
  width: 454px;
  height: 454px;
  background-color: white;
}
.chart {
  width: 300px;
  height: 300px;
}
// xxx.js
export default {
  data: {
    barData: [
      {
        fillColor: '#f07826',
        data: [763, 550, 551, 554, 731, 654, 525, 696, 595, 628],
      },
      {
        fillColor: '#cce5ff',
        data: [535, 776, 615, 444, 694, 785, 677, 609, 562, 410],
      },
      {
        fillColor: '#ff88bb',
        data: [673, 500, 574, 483, 702, 583, 437, 506, 693, 657],
      },
    ],
    barOps: {
      xAxis: {
        min: 0,
        max: 20,
        display: false,
        axisTick: 10
      },
      yAxis: {
        min: 0,
        max: 1000,
```

```
        display: false,
      },
    },
  }
}
```

图3-8 chart 组件 柱状图示例结果



3.3.2 image

图片组件，用来渲染展示图片。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-24 所示属性。

表3-24 image 组件属性说明

名称	类型	默认值	必填	说明
src	string	-	否	图片的路径，支持的图片格式包括 png、jpg。

说明

轻量级智能穿戴上，单个应用所有的图片资源总大小不得超过 8M。

样式

支持“3.1.2 通用样式”中所述样式。

事件

支持“3.1.3 通用事件”中所述事件。

3.3.3 image-animator

图片帧动画播放器。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-25 所示属性。

表3-25 image-animator 组件属性说明

名称	类型	默认值	必填	说明
images	Array<ImageFrame>	-	是	设置图片帧信息集合。每一帧的帧信息包含图片路径、图片大小和图片位置信息。目前支持以下图片格式：png、jpg。ImageFrame 的详细说明请见表 3-26。 说明 使用时需要使用数据绑定的方式，如 images = {{images}}，js 中声明相应变量：images: [{src: "/common/heart-rate01.png"}, {src: "/common/heart-rate02.png"}]。
iteration	number string	infinite	否	设置帧动画播放次数。number 表示固定次数，infinite 枚举表示无限次数播放。
reverse	boolean	false	否	设置播放顺序。

名称	类型	默认值	必填	说明
				• false：从第 1 张图片播放到最后 1 张图片。 • true：从最后 1 张图片播放到第 1 张图片。
fixedsize	boolean	true	否	设置图片大小是否固定为组件大小。 • true 表示图片大小与组件大小一致，此时设置图片的 width 、 height 、 top 和 left 属性是无效的。 • false 表示每一张图片的 width 、 height 、 top 和 left 属性都要单独设置。
duration	string	-	是	设置单次播放时长。单位支持 [s(秒) ms(毫秒)]，默认单位为 ms。 duration 为 0 时，不播放图片。 值改变只会在下一次循环开始时生效。
fillmode ⁵⁺	string	forwards	否	指定帧动画执行结束后的状态。可选项有： • none：恢复初始状态。 • forwards：保持帧动画结束时的状态（在最后一个关键帧中定义）。

表3-26 ImageFrame 说明

名称	类型	默认值	必填	说明
src	<uri>	-	是	图片路径。
width	<length>	0	否	图片宽度。

名称	类型	默认值	必填	说明
height	<length>	0	否	图片高度。
top	<length>	0	否	图片相对于组件左上角的纵向坐标。
left	<length>	0	否	图片相对于组件左上角的横向坐标。

样式

支持 “3.1.2 通用样式” 中所述样式。

事件

除支持 “3.1.3 通用事件” 中所述事件外，还支持如表 3-27 所示事件。

表3-27 image-animator 组件事件说明

名称	参数	说明
stop	-	帧动画结束时触发。

方法

表3-28 image-animator 组件方法说明

名称	参数	说明
start	-	开始播放图片帧动画。再次调用，重新从第 1 帧开始播放。
pause	-	暂停播放图片帧动画。
stop	-	停止播放图片帧动画。
resume	-	继续播放图片帧。
getState	-	获取播放状态。可能值有： - playing：播放中。 - paused：已暂停。 - stopped：已停止。

示例

```
<!-- xxx.html -->
<div class="container">
  <image-animator class="animator" ref="animator" images="{{frames}}" duration="1s" />
  <div class="btn-box">
    <input class="btn" type="button" value="start" @click="handleStart" />
    <input class="btn" type="button" value="stop" @click="handleStop" />
    <input class="btn" type="button" value="pause" @click="handlePause" />
    <input class="btn" type="button" value="resume" @click="handleResume" />
  </div>
</div>

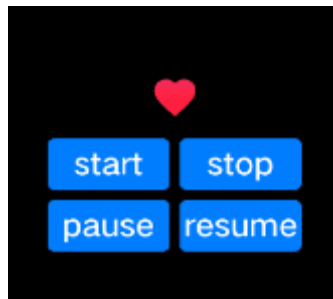
/* xxx.css */
.container {
  flex-direction: column;
  justify-content: center;
  align-items: center;
  left: 0px;
  top: 0px;
  width: 454px;
  height: 454px;
}
.animator {
  width: 70px;
  height: 70px;
}
.btn-box {
  width: 264px;
  height: 120px;
  flex-wrap: wrap;
  justify-content: space-around;
  align-items: center;
}
.btn {
  border-radius: 8px;
  width: 120px;
  margin-top: 8px;
}

//xxx.js
export default {
  data: {
    frames: [
```

```
{
  src: "/common/asserts/heart78.png",
},
{
  src: "/common/asserts/heart79.png",
},
{
  src: "/common/asserts/heart80.png",
},
{
  src: "/common/asserts/heart81.png",
},
{
  src: "/common/asserts/heart82.png",
},
{
  src: "/common/asserts/heart83.png",
},
{
  src: "/common/asserts/heart84.png",
},
{
  src: "/common/asserts/heart85.png",
},
{
  src: "/common/asserts/heart86.png",
},
{
  src: "/common/asserts/heart87.png",
},
{
  src: "/common/asserts/heart88.png",
},
{
  src: "/common/asserts/heart89.png",
},
{
  src: "/common/asserts/heart90.png",
},
{
  src: "/common/asserts/heart91.png",
},
{
```

```
src: "/common/asserts/heart92.png",
},
{
src: "/common/asserts/heart93.png",
},
{
src: "/common/asserts/heart94.png",
},
{
src: "/common/asserts/heart95.png",
},
{
src: "/common/asserts/heart96.png",
},
},
},
handleStart() {
this.$refs.animator.start();
},
handlePause() {
this.$refs.animator.pause();
},
handleResume() {
this.$refs.animator.resume();
},
handleStop() {
this.$refs.animator.stop();
},
};
```

图3-9 image-animator 组件示例结果



3.3.4 input

交互式组件，包括单选框，多选框，按钮。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-29 所示属性。

表3-29 input 组件属性说明

名称	类型	默认值	必填	说明
type	string	button	否	input 组件类型，可选值为 button，checkbox，radio。 button，checkbox，radio 不支持动态修改。可选值定义如下： • button：定义可点击的按钮。 • checkbox：定义多选框。 • radio：定义单选按钮，允许在多个拥有相同 name 值的选项中选中其中一个。
checked	boolean	false	否	当前组件是否选中，仅 type 为 checkbox 和 radio 生效。
name	string	-	否	input 组件的名称。
value	string	-	否	input 组件的 value 值，当类型为 radio 时必填且相同 name 值的选项该值唯一。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-30 所示样式。

表3-30 input 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#ffffff	否	单行输入框或者按钮的文本颜色。
font-size	<length>	30px	否	单行输入框或者按钮的文本尺寸。

事件

除支持“3.1.3 通用事件”中所述事件外，还支持如下事件：

- 当 input 类型为 checkbox、radio 时，支持如下事件：

表3-31 input 组件事件说明

名称	参数	说明
change	{ checked:true false }	checkbox 多选框或 radio 单选框的 checked 状态发生变化时触发该事件。

3.3.5 marquee

跑马灯组件，用于展示一段单行滚动的文字。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-32 所示属性。

表3-32 marquee 组件属性说明

名称	类型	默认值	必填	说明
scrollamount	number	6	否	跑马灯每次滚动时移动的最大长度。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-33 所示样式。

表3-33 marquee 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#ffffff	否	设置跑马灯中文字的文本颜色。
font-size	<length>	30	否	设置跑马灯中文字的文本尺寸。
font-family	string	HYQiHei-65S	否	字体。目前仅支持 HYQiHei-65S 字体。

事件

支持“3.1.3 通用事件”中所述事件。

3.3.6 picker-view

嵌入页面的滑动选择器。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-34 所示属性。

表3-34 picker-view 组件属性说明

名称	类型	默认值	必填	说明
type	string	text	否	设置滑动选择器的类型，该属性不支持动态修改，可选项有： - text：文本选择器。 - time：时间选择器。
loop	boolean	true	否	是否开启循环滑动。不支持动态修改。

不同的滑动选择器的类型还支持不同的属性。

- 类型为文本选择器时，type=text，支持如表 3-35 所示属性。

表3-35 文本选择器属性说明

名称	类型	默认值	必填	说明
range	Array	-	否	设置文本选择器的取值范围。 说明 使用时需要使用数据绑定的方式，如 range = {{data}}，js 中声明相应变量： data: ["15", "20", "25"]。
selected	string	0	否	设置文本选择器的默认选择值，该 值需要为 range 的索引。

- 类型为时间选择器时，type=time，支持如表 3-36 所示属性。

表3-36 时间选择器属性说明

名称	类型	默认值	必填	说明
selected	string	00:00	否	设置时间选择器的默认取值，格式 为 HH:mm；

样式

除支持 “3.1.2 通用样式” 中所述样式外，还支持如表 3-37 所示样式。

表3-37 picker-view 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#808080	否	候选项字体颜色。
font-size	<length>	30px	否	候选项字体尺寸，类型 length， 单位：px。

名称	类型	默认值	必填	说明
selected-color	<color>	#ffffff	否	选中项字体颜色。
selected-font-size	<length>	38px	否	选中项字体尺寸，类型 length，单位：px。
font-family	string	HYQiHei-65S	否	选项字体类型。 字体。目前仅支持 HYQiHei-65S 字体。

事件

仅支持如下事件：

- 类型为文本选择器时，type=text，支持如表 3-38 所示事件。

表3-38 文本选择器事件说明

名称	参数	说明
change	{ newValue: newValue, newSelected: newSelected }	文本选择器选定值后触发该事件。

- 类型为时间选择器时，type=time，支持如表 3-39 所示事件。

表3-39 时间选择器事件说明

名称	参数	说明
change	{ hour: hour, minute: minute }	时间选择器选定值后触发该事件。

方法

不支持。

示例

```
<!-- xxx.html -->
<div class="container" @swipe="handleSwipe">
```

```
<text class="title">
    Selected: {{time}}
</text>
<picker-view class="time-picker" type="time" selected="{{defaultTime}}"
@change="handleChange"></picker-view>
</div>
/* xxx.css */
.container {
    flex-direction: column;
    justify-content: center;
    align-items: center;
    left: 0px;
    top: 0px;
    width: 454px;
    height: 454px;
}
.title {
    font-size: 30px;
    text-align: center;
}
.time-picker {
    width: 500px;
    height: 400px;
    margin-top: 20px;
}
/* xxx.js */
export default {
    data: {
        defaultTime: "",
        time: "",
    },
    onInit() {
        this.defaultTime = this.now();
    },
    handleChange(data) {
        this.time = this.concat(data.hour, data.minute);
    },
    now() {
        const date = new Date();
        const hours = date.getHours();
        const minutes = date.getMinutes();
        return this.concat(hours, minutes);
    },
}
```

```
fill(value) {
    return (value > 9 ? "" : "0") + value;
},

concat(hours, minutes) {
    return `${this.fill(hours)}:${this.fill(minutes)}`;
},
}
```

图3-10 picker-view 组件示例结果



3.3.7 progress

进度条，用于显示内容加载或操作处理进度。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-40 所示属性。

表3-40 progress 组件属性说明

名称	类型	默认值	必填	说明
type	string	horizontal	否	设置进度条的类型，该属性不支持动态修改，可选值为： - horizontal：线性进度条。 - arc：弧形进度条。

不同类型的进度条还支持不同的属性。

- 类型为 horizontal 时，type=horizontal，支持如表 3-41 所示属性。

表3-41 线性进度条属性说明

名称	类型	默认值	必填	说明
percent	number	0	否	当前进度。取值范围为 0 ~ 100。

- 类型为 arc 时，type=arc，支持如表 3-42 所示属性。

表3-42 弧形进度条属性说明

名称	类型	默认值	必填	说明
percent	number	0	否	当前进度。取值范围为 0 ~ 100。

样式

除支持 “3.1.2 通用样式” 中所述样式外，还支持如下样式。

- 类型为 horizontal 时，type=horizontal，支持如表 3-43 所示样式。

表3-43 线性进度条样式说明

名称	类型	默认值	必填	说明
color	<color>	#6b9ac7	否	设置进度条的颜色。
stroke-width	<length>	32px	否	设置进度条的宽度。

- 类型为 arc 时，type=arc，支持如表 3-44 所示样式。

表3-44 弧形进度条样式说明

名称	类型	默认值	必填	说明
color	<color>	#5ea1ff	否	弧形进度条的颜色。

名称	类型	默认值	必填	说明
background-color	<color>	rgba(255, 255, 255, 0.15)	否	弧形进度条的背景色。
stroke-width	<length>	32px	否	弧形进度条的宽度。 说明 进度条宽度越大，进度条越靠近圆心，进度条始终在半径区域内。
start-angle	<deg>	240	否	弧形进度条起始角度，以时钟 0 点为基线，取值范围为 0 到 360（顺时针）。
total-angle	<deg>	240	否	弧形进度条总长度，范围为-360 到 360，负数标识起点到终点为逆时针。
center-x	<length>	弧形进度条宽度的一半	否	弧形进度条中心位置，（坐标原点为组件左上角顶点）。该样式需要和 center-y 和 radius 一起使用。
center-y	<length>	弧形进度条高度的一半	否	弧形进度条中心位置，（坐标原点为组件左上角顶点）。该样式需要和 center-x 和 radius 一起使用。
radius	<length>	弧形进度条宽高最小值的一半	否	弧形进度条半径，该样式需要和 center-x 和 center-y 一起使用。

事件

支持 “3.1.3 通用事件” 中所述事件。

3.3.8 qrcode

生成并显示二维码。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-45 所示属性。

表3-45 qrcode 组件属性说明

名称	类型	默认值	必填	说明
value	string	-	是	用来生成二维码的内容。 最大长度为 256。
type	string	rect	否	二维码类型。 rect：矩形二维码。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-46 所示样式。

表3-46 qrcode 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#000000	否	二维码颜色。
background-color	<color>	#ffffff	否	二维码背景颜色。

说明

- width 和 height 不一致时，取二者较小值作为二维码的边长。且最终生成的二维码居中显示。
- width 和 height 的最小值与 value 长度成正比，请参见表 3-47。

表3-47 二维码最小尺寸与 value 长度关系

value 长度	≤17	≤32	≤53	≤78	≤106	≤134	≤154	≤192	≤230
二维	21×	25×	29×	33×	37×	41×	45×	49×	53×

value 长度	≤17	≤32	≤53	≤78	≤106	≤134	≤154	≤192	≤230
码最 小尺 寸	21	25	29	33	37	41	45	49	53

事件

支持 “3.1.3 通用事件” 中所述事件。

示例

```
/* xxx.css */
.qrcode_class{
  width: 53px;
  height: 53px;
}
.container{
  width: 100%;
  height: 100%;
  justify-content: center;
  align-items: center;
}
<!-- xxx.html -->
<div class="container">
<qrcode class="qrcode_class" value="https://example.com"/>
</div>
```

3.3.9 slider

滑动条组件，用来快速调节设置值，如音量、亮度等。

子组件

不支持。

属性

除支持 “3.1.1 通用属性” 中所述属性外，还支持如表 3-48 所示属性。

表3-48 slider 组件属性说明

名称	类型	默认值	必填	说明
min	number	0	否	滑动选择器的最小值。
max	number	100	否	滑动选择器的最大值。
value	number	0	否	滑动选择器的初始值。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-49 所示样式。

表3-49 slider 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#000000	否	滑动条的背景颜色。
selected-color	<color>	#ffffff	否	滑动条的已选择颜色。
direction	column row	row	否	滑动条方向，可选项有： column：垂直方向。 row：水平方向。

事件

除支持“3.1.3 通用事件”中所述事件外，还支持如表 3-50 所示事件。

表3-50 slider 组件事件说明

名称	参数	说明
change	表 3-51	选择值发生变化时触发该事件。

表3-51 ChangeEvent

属性	类型	说明
----	----	----

属性	类型	说明
progress(deprecated)	string	当前 slider 的进度值。
value	number	当前 slider 的值。

3.3.10 switch

开关选择器，通过开关开启或关闭某个功能。

子组件

不支持。

属性

除支持“3.1.1 通用属性”中所述属性外，还支持如表 3-52 所示属性。

表3-52 switch 组件属性说明

名称	类型	默认值	必填	说明
checked	boolean	false	否	是否选中。

样式

支持“3.1.2 通用样式”中所述样式。

事件

除支持“3.1.3 通用事件”中所述事件外，还支持如表 3-53 所示事件。

表3-53 switch 组件事件说明

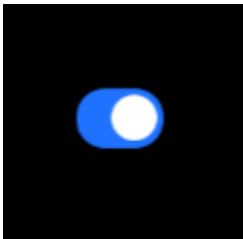
名称	参数	说明
change	{ checked: checkedValue }	选中状态改变时触发该事件。

示例

```
<!-- xxx.html -->
<div class="container">
  <switch checked="true" @change="switchChange">
    </switch>
</div>
/* xxx.css */
.container {
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
}
switch{
  width: 100px;
  height: 100px;
}
// xxx.js

export default {
  switchChange(e){
    console.log(e.checked);
  }
}
```

图3-11 switch 组件示例说明



3.3.11 text

文本，用于呈现一段信息。

子组件

不支持。

属性

支持“3.1.1 通用属性”中所述属性。

样式

除支持“3.1.2 通用样式”中所述样式外，还支持如表 3-54 所示样式。

表3-54 text 组件样式说明

名称	类型	默认值	必填	说明
color	<color>	#ffffff	否	设置文本的颜色。
font-size	<length>	30px	否	设置文本的尺寸。 目前仅支持 30px 和 38px 两个字体大小。
letter-spacing	<length>	2px	否	设置文本的字符间距。
text-alignright	string	left	否	设置文本的文本对齐方式，可选值为： - left：文本水平方向左对齐。 - center：文本水平方向居中对齐。 - right：文本水平方向右对齐。
text-align-vertical	string	top	否	设置文本的文本对齐方式，可选值为： - top：文本垂直方向顶部对齐。 - center：文本垂直方向居中对齐。 - bottom：文本垂直方向底部对齐。
text-overflow	string	clip	否	可选值为： - clip：将文本根据父容器大小进行裁剪显示。 - ellipsis：根据父容器大小显示，

名称	类型	默认值	必填	说明
				显示不下的文本用省略号代替。
font-family	string	HYQiHei-65S	否	字体。目前仅支持 HYQiHei-65S 字体。

事件

支持 [“3.1.3 通用事件”](#) 中所述事件。

示例

```
<!-- xxx.html -->
<div class="container">

  <text class="title">
    Hello {{ title }}
  </text>
</div>

/* xxx.css */
.container {
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
}
.title {
  font-size: 30px;
  text-align: center;
  width: 100px;
}

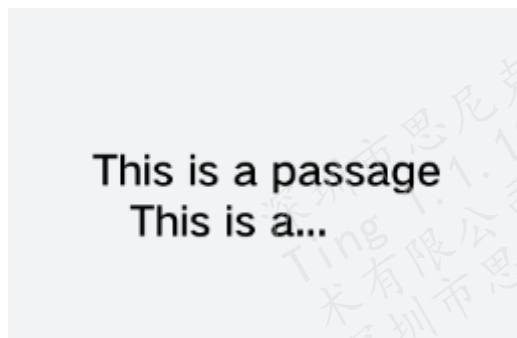
// xxx.js
export default {
  data: {
    title: 'World'
  }
}
```

图3-12 text 组件-示例结果 1



```
<!-- xxx.html -->
<div class="container">
  <text class="text1">
    This is a passage
  </text>
  <text class="text2">
    This is a passage
  </text>
</div>
/* xxx.css */
.container {
  width: 100%;
  height: 100%;
  flex-direction: column;
  align-items: center;
  background-color: #F1F3F5;
  justify-content: center;
}
.text1{
  color: black;
}
.text2{
  width: 200px;
  height: 46px;
  color: black;
  max-lines: 1;
  text-overflow: ellipsis;
  text-align: middle;
  line-height: 40px;
}
```

图3-13 text 组件-示例结果 2



3.4 扩展组件

1. ACE 上扩展组件。

- 在 components 路径下添加自定义组件的具体实现，自定义组件继承自 component.h。

components 路径：foundation/ace/ace_engine_lite/components

```
class CanvasComponent final : public Component {  
    ...  
};
```

- 在 keys.h 中添加组件的 key 值及名称。

keys.h 路径：foundation/ace/ace_engine_lite/base/keys.h

```
enum {  
    ...  
    KEYWORD(CANVAS, canvas) // canvas component  
}
```

- 在 component_factory.h 中增加新增组件对应的构造函数。

component_factory.h 目录：

foundation/ace/ace_engine_lite/components/component_factory.h

```
static Component* CreateComponent(...)  
{  
    ...  
    switch (componentNameId) {  
        ...  
        case K_QRCODE:  
            component = new CanvasComponent(options, children, styleManager);  
            break;
```

87

4 接口

- 4.1 通用规则
- 4.2 通用错误码
- 4.3 基本功能
- 4.4 扩展接口

4.1 通用规则

同步

同步方法调用后必须等到方法结果返回后才能继续后续的行为，返回值可以是任意类型。

示例：

```
var info = app.getInfo();
console.log(JSON.stringify(info));
```

异步

异步方法调用整个过程不会阻碍调用者的工作。业务执行完成后会调用开发者提供的回调函数，异步接口支持的回调函数如表 4-1 所示。

表4-1 异步接口支持的回调函数

回调函数	参数名	类型	返回值	说明
success	data	any	可选，返回值可以是任	在执行成功时触发。

回调函数	参数名	类型	返回值	说明
			意类型。	
fail	data	any	错误信息内容，一般是字符串，也可能是其他类型。	在执行失败时触发。
	code	number	错误代码，请参见“4.2 通用错误码”。	
cancel	data	any	一般无内容。	在用户取消时触发。
complete	-	-	-	在执行完成时触发。

说明

- success、fail、cancel 和 complete 四个回调函数是否支持请参考具体接口描述。
- success、fail 和 cancel 三个回调函数的触发是互斥的，即会且只会有一个回调函数被触发，触发任意一个都会再次调用 complete 回调。

示例：

```
device.getInfo({
  success: function(data) {
    console.log('Device information obtained successfully. Device brand:' + data.brand);
  },
  fail: function(data, code) {
    console.log('Failed to obtain device information. Error code:' + code + '; Error information: '
+ data);
  },
});
```

订阅

订阅接口不会立即返回结果，开发者要在参数中设置相应的回调函数，该回调函数会在完成时或者事件变化时进行回调；可以执行多次。

表4-2 订阅接口支持的回调函数

回调函数	参数名	类型	返回值	说明
success	data	any	返回值可以是任意类型	接口调用成功或事件变

回调函数	参数名	类型	返回值	说明
				更时触发，可能会触发多次。
fail	data	any	错误信息内容，一般是字符串，也可能是其他类型。	在执行失败时触发。一旦触发该回调函数，success 不会再次被调用，接口调用结束。
	code	number	错误代码，请参见“4.2 通用错误码”。	

4.2 通用错误码

如表 4-3 所示提供公共的错误码。其中，错误码 200 为系统通用错误码，所有系统未知异常发生时抛出，如框架申请内存空间失败等情况。

表4-3 通用错误码

code	含义
200	通用错误。
202	参数错误。
300	I/O 错误。

4.3 基本功能

4.3.1 应用上下文

导入模块

```
import app from '@system.app';
```

app.getInfo

获取当前应用配置文件中声明的信息：getInfo()

- 返回值

表4-4 AppResponse

参数名	类型	说明
appName	string	表示应用的名称。
versionName	string	表示应用的版本名称。
versionCode	number	表示应用的版本号。

- 示例

```
var info = app.getInfo();  
console.log(JSON.stringify(info));
```

app.terminate

退出当前 Ability：terminate(): void

示例：

```
app.terminate();
```

4.3.2 日志打印

导入模块

无需导入。

console.debug

打印 debug 级别的日志信息：debug(message: string): void

- 参数

参数名	类型	必填	说明
message	string	是	表示要打印的文本信息。

console.log

打印 log 级别的日志信息：log(message: string): void

- 参数

参数名	类型	必填	说明
message	string	是	表示要打印的文本信息。

console.info

打印 info 级别的日志信息：info(message: string): void

- 参数

参数名	类型	必填	说明
message	string	是	表示要打印的文本信息。

console.warn

打印 warn 级别的日志信息：warn(message: string): void

- 参数

参数名	类型	必填	说明
message	string	是	表示要打印的文本信息。

console.error

打印 error 级别的日志信息：error(message: string): void

- 参数

参数名	类型	必填	说明
message	string	是	表示要打印的文本信息。

示例

```
var versionCode = 1;
console.info('Hello World. The current version code is ' + versionCode);
```

4.3.3 页面路由

导入模块

```
import router from '@system.router';
```

router.replace

用应用内的某个页面替换当前页面并销毁被替换的页面：replace(Object): void

参数

参数名	类型	必填	说明
uri	string	是	目标页面的 uri，可以是以下的两种格式： - 页面绝对路径，由配置文件中 pages 列表提供，例如： - pages/index/index - pages/detail/detail - 特殊值，如果 uri 的值是 "/"，则跳转到首页。
params	Object	否	跳转时要同时传递到目标页面的数据，跳转到目标页面后，参数可以在页面中直接使用，如 this.data1（data1 为跳转时 params 参数中的 key 值）。如果目标页面中已有该字段，则其值会被传入的字段值覆盖。

示例

```
// 在当前页面中
export default {
  replacePage() {
    router.replace({
      uri: 'pages/detail/detail',
      params: {
        data1: 'message',
      },
    });
  }
}
```

```
    });  
  }  
}  
// 在detail页面中  
export default {  
  data: {  
    data1: 'default'  
  },  
  onInit() {  
    console.info('showData1:' + this.data1)  
  }  
}
```

4.3.4 应用配置

导入模块

```
import configuration from '@system.configuration';
```

configuration.getLocale

获取应用当前的语言和地区：getLocale()，默认与系统的语言和地区同步。

- 返回值

表4-5 LocaleResponse

参数名	类型	说明
language	string	语言。例如：zh。
countryOrRegion	string	国家或地区。例如：CN。
dir	string	文字布局方向。取值范围： - ltr：从左到右； - rtl：从右到左。

- 示例

```
const localeInfo = configuration.getLocale();  
console.info(localeInfo.language);
```


4.3.5 定时器

导入模块

无需导入。

权限列表

无

setTimeout

设置一个定时器，该定时器在定时器到期后执行一个函数：

setTimeout(handler[,delay[,...args]]): number

- 参数

参数名	类型	必填	说明
handler	Function	是	定时器到期后执行函数。
delay	number	否	延迟的毫秒数，函数的调用会在该延迟之后发生。如果省略该参数，delay 取默认值 0，意味着“马上”执行，或尽快执行。
...args	Array<any>	否	附加参数，一旦定时器到期，它们会作为参数传递给 handler。

- 返回值

类型	说明
number	timeout 定时器的 ID。

- 示例

```
var timeoutID = setTimeout(function() {  
    console.log('delay 1s');  
}, 1000);
```

clearTimeout

clearTimeout(timeoutID: number): void

取消了先前通过调用 setTimeout()建立的定时器。

- 参数

参数名	类型	必填	说明
timeoutID	number	是	要取消定时器的 ID, 是由 setTimeout() 返回的。

- 示例

```
var timeoutID = setTimeout(function() {  
    console.log('do after 1s delay.');
```

```
}, 1000);
```

```
clearTimeout(timeoutID);
```

setInterval

setInterval(handler[, delay[, ...args]]): number

重复调用一个函数, 在每次调用之间具有固定的时间延迟。

- 参数

参数名	类型	必填	说明
handler	Function	是	要重复调用的函数。
delay	number	否	延迟的毫秒数 (一秒等于 1000 毫秒), 函数的调用会在该延迟之后发生。
...args	Array<any>	否	附加参数, 一旦定时器到期, 他们会作为参数传递给 handler。

- 返回值

类型	说明
number	intervalID 重复定时器的 ID。

- 示例

```
var intervalID = setInterval(function() {
    console.log('do very 1s.');
```

```
}, 1000);
```

clearInterval

clearInterval(intervalID: number): void

可取消先前通过 setInterval() 设置的重复定时任务。

- 参数

参数名	类型	必填	说明
intervalID	number	是	要取消的重复定时器的 ID，是由 setInterval() 返回的。

- 示例

```
var intervalID = setInterval(function() {
    console.log('do very 1s.');
```

```
}, 1000);
```

```
clearInterval(intervalID);
```

4.3.6 网络

导入模块

```
import http from '@ohos.net.http';
```

http.createHttp

创建一个 HTTP 请求，支持发起请求、中断请求。

- 返回值

表4-6 HttpRequest

返回值名称	类型	说明
HttpRequest	Object	HTTP 请求对象，支持 request、requestInStream、on、off、destroy 等方

返回值名称	类型	说明
		法。

- 示例

```
let httpRequest = http.createHttp();
```

httpRequest.request

向 URL 地址，发起 HTTP 请求，通过注册异步回调方法，处理响应结果。

- 参数

表4-7 request

参数名称	类型	说明
url	string	请求地址。
HttpRequestOptions	Object	请求参数结构如下： { method: String, header: String, clientCert: String, extraData: null, } 其中 method 支持 GET、POST、PUT、DELETE 等常用方法。
AsyncCallback	Function	参数 responseCode，类型 number：请求返回的响应码。 参数 data，类型 string ArrayBuffer：请求返回的数据内容。

- 示例

```
let httpRequest = http.createHttp();
httpRequest.request(
  "http://www.foo.com/img/test.png",
  {
    method: "GET",
    header: null,
    clientCert: null,
    extraData: null,
```

```

    },
    (responseCode, data) => {
        console.info('Result CallBack');
        if (responseCode == 200) {
            console.info('Http request Success: responseCode: ' +
JSON.stringify(responseCode));
            console.info('Http request Success: data: ' + JSON.stringify(data.result));
        } else {
            console.info('Http request Fail: responseCode: ' +
JSON.stringify(responseCode));
            console.info('Http request Fail: data: ' + JSON.stringify(data.result));
        }
    }
});

```

4.4 扩展接口

1. 在 ohos_module_config.h 中定义模块名称及初始化模块入口。

ohos_module_config.h 目录：

foundation/ace/ace_engine_lite/modules/ohos_module_config.h

```

const Module OHOS_MODULES[] = {
#ifdef FEATURE_MODULE_AUDIO
    {"audio", InitAudioModule},
#endif // FEATURE_MODULE_AUDIO
}

```

2. 模块定义。

```

// AudioModule.h
class AudioModule final : public MemoryHeap {
public:
    // 模块结束时释放资源
    static void OnTerminate();

    // 自定义方法
    static JSValue Play();
    static JSValue Pause();
    static JSValue SrcSetter();
    ...
}

// 模块初始化入口
void InitAudioModule(JSValue exports);

```

```
// AudioModule.cpp
void InitAudioModule(JSValue exports)
{
    InitAdioPlayer();

    // 绑定模块终止方法
    JSI::SetOnTerminate(exports, AudioModule::OnTerminate);

    // 绑定方法到js
    JSI::SetModuleAPI(exports, "play", AudioModule::Play);
    JSI::SetModuleAPI(exports, "pause", AudioModule::Pause);
    JSI::SetModuleAPI(exports, "stop", AudioModule::Stop);

    ...

    // 绑定属性到js
    AudioModule::DefineProperty(exports, "src", AudioModule::SrcGetter,
    AudioModule::SrcSetter);

    ...
}
```

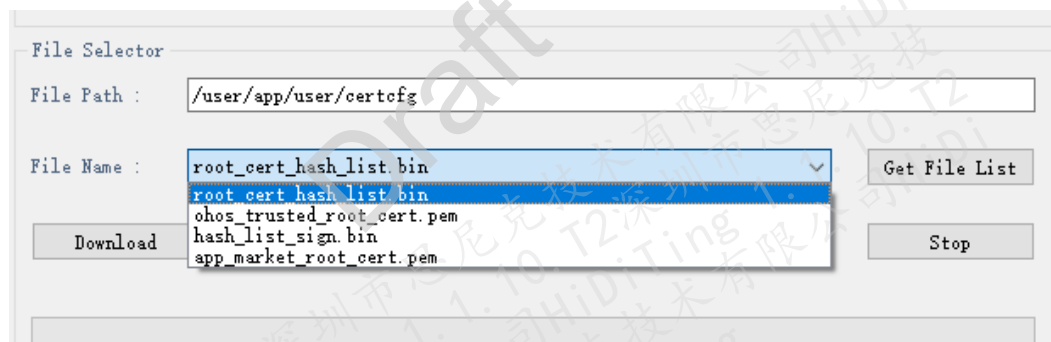
3. 在 Deveco Studio 中使用。

```
// index.js
import player from '@system.audio'
player.src = "test.mp3";
player.play();
```

5 OH 应用市场

穿戴设备配套的手机管理软件，在接入应用市场 (<https://www.openwearplay.com>) 后，可以安装管理 OpenHarmony 穿戴应用市场中的 JsApp；应用安全安装需要使用根证书进行证书链的完整性校验；

穿戴设备端需上传证书链的根证书文件以及根证书文件的哈希列表、哈希列表签名文件到设备的 `"/user/app/user/certcfg"` 目录下，如下图所示：



- `ohos_trusted_root_cert.bin`：OpenHarmony 社区根证书文件。
- `app_market_root_cert.pem`：穿戴应用市场根证书文件。
- `root_cert_hash_list.bin`：包含根证书文件名及根证书内容 SHA256 值的哈希列表文件。
- `hash_list_sign.bin`：使用穿戴市场私钥对哈希列表进行签名的签名文件，验签的公钥已预置在镜像内。

须知

根证书可以只传一个，也支持可扩展，如上同时支持 OpenHarmony 社区签名的应用也支持穿戴应用市场签名的应用；如果不推上述文件，开启应用验签的情况下，设备只能安装 OpenHarmony 社区签名的应用。

A 类型说明

长度类型

名称	类型定义	说明
length	string number	用于描述尺寸单位。 - 输入为 number 类型时，使用 px 单位。 - 输入为 string 类型时，需要显式指定像素单位，当前支持的像素单位有：px（逻辑尺寸单位）。
percentage	string	百分比尺寸单位，如 “50%”。

颜色类型

表A-1 当前支持的颜色枚举

名称	类型定义	说明
color	string 下表中枚举名	用于描述颜色信息。 字符串格式如下： 'rgb(255, 255, 255)' 'rgba(255, 255, 255, 1.0)' - HEX 格式：'#rrggbb', '#aarrggbb' - 枚举格式：'black', 'white'。 说明 JS 脚本中不支持颜色枚举格式。

枚举名称	十六进制颜色码	颜色
aliceblue	#f0f8ff	
antiquewhite	#faebd7	
aqua	#00ffff	
aquamarine	#7fffd4	
azure	#f0ffff	
beige	#f5f5dc	
bisque	#ffe4c4	
black	#000000	
blanchedalmond	#ffeacd	
blue	#0000ff	
blueviolet	#8a2be2	
brown	#a52a2a	
burlywood	#deb887	
cadetblue	#5f9ea0	
chartreuse	#7fff00	
chocolate	#d2691e	
coral	#ff7f50	
cornflowerblue	#6495ed	
cornsilk	#fff8dc	
crimson	#dc143c	
cyan	#00ffff	
darkblue	#00008b	
darkcyan	#008b8b	
darkgoldenrod	#b8860b	
darkgray	#a9a9a9	
darkgreen	#006400	

枚举名称	十六进制颜色码	颜色
darkgrey	#a9a9a9	
darkkhaki	#bdb76b	
darkmagenta	#8b008b	
darkolivegreen	#556b2f	
darkorange	#ff8c00	
darkorchid	#9932cc	
darkred	#8b0000	
darksalmon	#e9967a	
darkseagreen	#8fbc8f	
darkslateblue	#483d8b	
darkslategray	#2f4f4f	
darkslategrey	#2f4f4f	
darkturquoise	#00ced1	
darkviolet	#9400d3	
deeppink	#ff1493	
deepskyblue	#00bfff	
dimgray	#696969	
dimgrey	#696969	
dodgerblue	#1e90ff	
firebrick	#b22222	
floralwhite	#fffaf0	
forestgreen	#228b22	
fuchsia	#ff00ff	
gainsboro	#dcdcdc	
ghostwhite	#f8f8ff	
gold	#ffd700	
goldenrod	#daa520	

枚举名称	十六进制颜色码	颜色
gray	#808080	
green	#008000	
greenyellow	#adff2f	
grey	#808080	
honeydew	#f0fff0	
hotpink	#ff69b4	
indianred	#cd5c5c	
indigo	#4b0082	
ivory	#fffff0	
khaki	#f0e68c	
lavender	#e6e6fa	
lavenderblush	#fff0f5	
lawngreen	#7cfc00	
lemonchiffon	#fffacd	
lightblue	#add8e6	
lightcoral	#f08080	
lightcyan	#e0ffff	
lightgoldenrodyellow	#fafad2	
lightgray	#d3d3d3	
lightgreen	#90ee90	
lightpink	#ffb6c1	
lightsalmon	#ffa07a	
lightseagreen	#20b2aa	
lightskyblue	#87cefa	
lightslategray	#778899	
lightslategrey	#778899	
lightsteelblue	#b0c4de	

枚举名称	十六进制颜色码	颜色
lightyellow	#ffffe0	
lime	#00ff00	
limegreen	#32cd32	
linen	#faf0e6	
magenta	#ff00ff	
maroon	#800000	
mediumaquamarine	#66cdaa	
mediumblue	#0000cd	
mediumorchid	#ba55d3	
mediumpurple	#9370db	
mediumseagreen	#3cb371	
mediumslateblue	#7b68ee	
mediumspringgreen	#00fa9a	
mediumturquoise	#48d1cc	
mediumvioletred	#c71585	
midnightblue	#191970	
mintcream	#f5fffa	
mistyrose	#ffe4e1	
moccasin	#ffe4b5	
navajowhite	#ffdead	
navy	#000080	
oldlace	#fdf5e6	
olive	#808000	
olivedrab	#6b8e23	
orange	#ffa500	
orangered	#ff4500	
orchid	#da70d6	

枚举名称	十六进制颜色码	颜色
palegoldenrod	#eee8aa	
palegreen	#98fb98	
paleturquoise	#afeeee	
palevioletred	#db7093	
papayawhip	#ffefd5	
peachpuff	#ffdab9	
peru	#cd853f	
pink	#ffc0cb	
plum	#dda0dd	
powderblue	#b0e0e6	
purple	#800080	
rebeccapurple	#663399	
red	#ff0000	
rosybrown	#bc8f8f	
royalblue	#4169e1	
saddlebrown	#8b4513	
salmon	#fa8072	
sandybrown	#f4a460	
seagreen	#2e8b57	
seashell	#fff5ee	
sienna	#a0522d	
silver	#c0c0c0	
skyblue	#87ceeb	
slateblue	#6a5acd	
slategray	#708090	
slategrey	#708090	
snow	#ffffa	

枚举名称	十六进制颜色码	颜色
springgreen	#00ff7f	
steelblue	#4682b4	
tan	#d2b48c	
teal	#008080	
thistle	#d8Bfd8	
tomato	#ff6347	
turquoise	#40e0d0	
violet	#ee82ee	
wheat	#f5deb3	
white	#ffffff	
whitesmoke	#f5f5f5	
yellow	#ffff00	
yellowgreen	#9acd32	